

DOSSIER DE PROJET PROFESSIONNEL Titre Professionnel

Développeur Web et Web Mobile

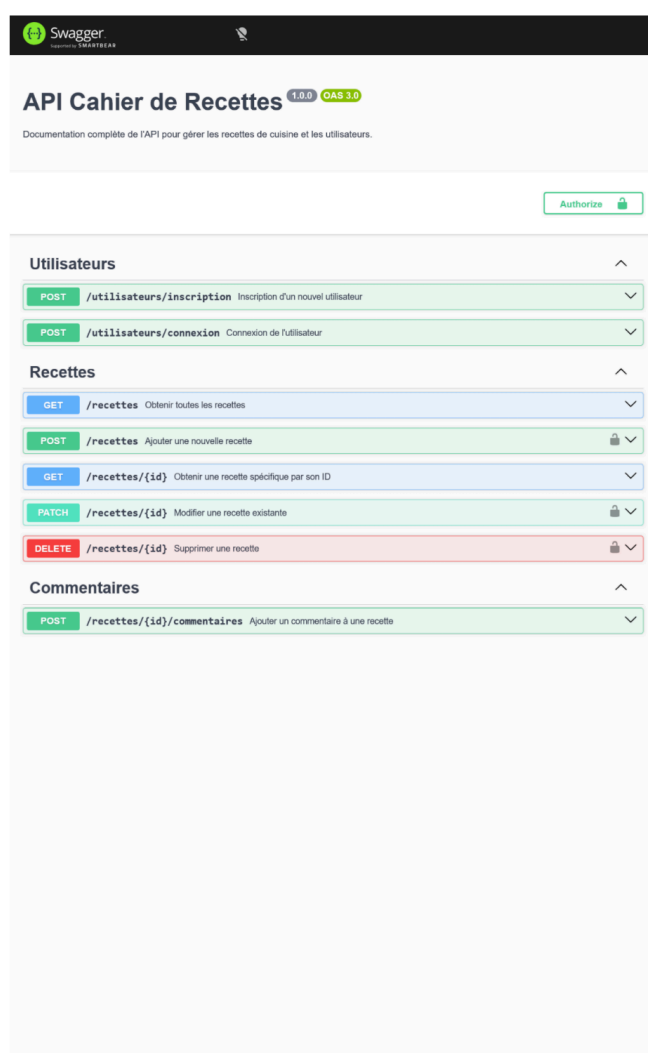
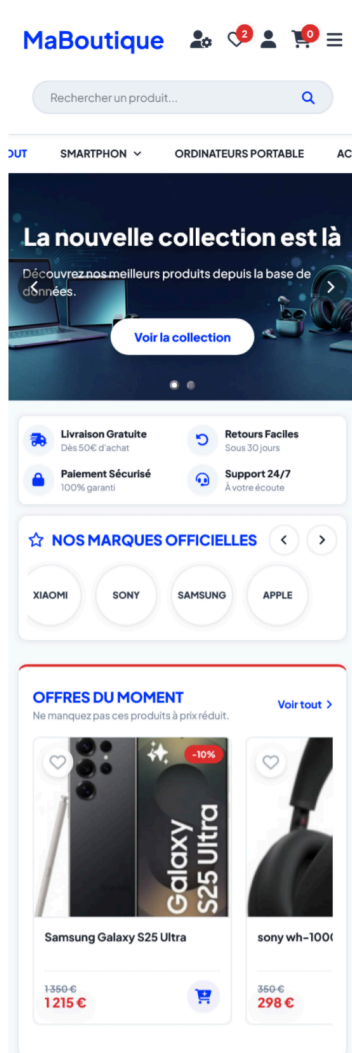
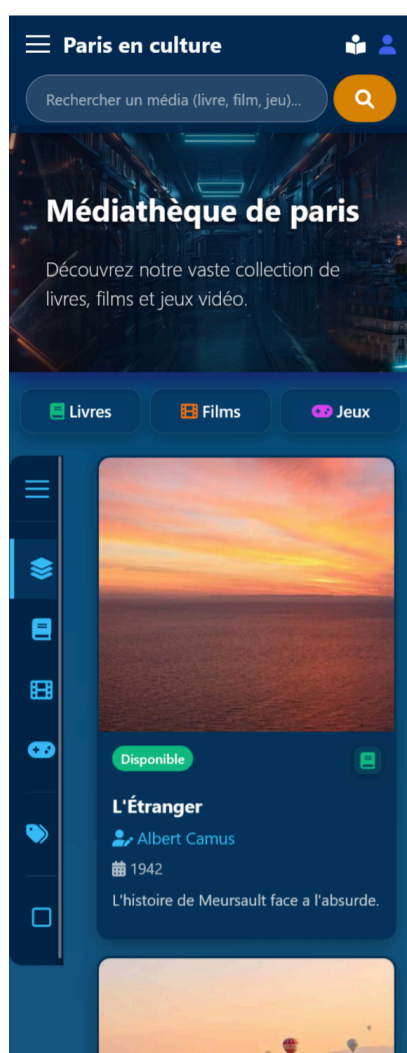
MÉDIATHÈQUE DE PARIS (Focus Front-End)

BOUTIQUE EN LIGNE (Focus Back-End & Sécurité)

CAHIER DE RECETTES (API REST Node.js)

Présenté par : **Jawad Zafari**

2025 - 2026 - Ecole La Plateforme (Paris)



SOMMAIRE

1. INTRODUCTION GÉNÉRALE.....	3
1.1. Contexte du projet et stratégie.....	3
1.2. Objectifs des applications.....	3
2.1. Expression des besoins.....	4
2.2. Rôles des utilisateurs.....	4
3. CONCEPTION ET INTÉGRATION UI/UX.....	6
3.1. Maquettes et charte graphique.....	6
3.2. Intégration et Responsive Design.....	7
3.3. Accessibilité Web et SEO.....	9
4. DYNAMISME ET JAVASCRIPT.....	10
4.1. Requêtes asynchrones et recherche en direct.....	10
5. ARCHITECTURE ET BASE DE DONNÉES.....	12
5.1. L'architecture MVC en PHP natif.....	12
5.2. Modélisation des données avec MySQL.....	12
5.2.1. Dictionnaire de Données.....	13
5.3. Connexion sécurisée avec PDO.....	14
6. FONCTIONNALITÉS ET ADMINISTRATION.....	15
6.1. Le tunnel d'achat et la gestion du panier.....	15
6.2. Panneau d'administration et opérations CRUD.....	16
7. SÉCURITÉ DE L'APPLICATION.....	17
7.1. Protection contre les failles CSRF et XSS.....	17
7.2. Authentification et gestion des sessions.....	18
7.3. Conformité RGPD et Protection des Données.....	19
7.4. Plan de Tests et Jeux d'Essai.....	20
8. GESTION DE PROJET ET VERSIONING.....	21
8.1. Workflow avec Git et GitHub.....	21
8.2. Organisation du travail en équipe.....	21
9. CAHIER DE RECETTES COLLABORATIF.....	22
9.1. Présentation du projet.....	22
9.2. Architecture et Base de données avec MongoDB.....	22
9.3. Sécurité et Authentification avec les jetons JWT.....	24
9.4. Documentation technique avec Swagger.....	25
9.5 - Phase de Tests et Validation avec Postman.....	26
10. CONCLUSION ET PERSPECTIVES.....	27
10.1. Bilan de la formation à La Plateforme.....	27
10.2. Compétences acquises.....	28
10.3. Évolutions futures des projets.....	28
10.4. Déploiement et Mise en Production.....	29
10.5. Méthodologie de déploiement appliquée.....	29
10.6. La Veille Technologique.....	31

1. INTRODUCTION GÉNÉRALE

1.1. Contexte du projet et stratégie

Dans le cadre de ma formation pour le Titre Professionnel Développeur Web et Web Mobile, abrégé DWWM, j'ai choisi de concevoir et de présenter trois projets distincts. Cette approche stratégique me permet de démontrer l'ensemble des compétences exigées pour le CCP 1 en développement Front-End et le CCP 2 en développement Back-End :

- Médiathèque de Paris : Ce projet met en valeur mes compétences en développement Front-End, notamment l'intégration web, le design UI/UX et la programmation dynamique avec JavaScript.
- MaBoutique : Ce projet valide mes compétences en développement Back-End. Il illustre ma maîtrise du langage PHP, de l'architecture MVC, de la gestion des bases de données relationnelles et de la sécurité web.
- Cahier de Recettes : Ce projet démontre ma capacité à concevoir une API RESTful moderne et sécurisée en utilisant l'environnement Node.js et une base de données NoSQL.



1.2. Objectifs des applications

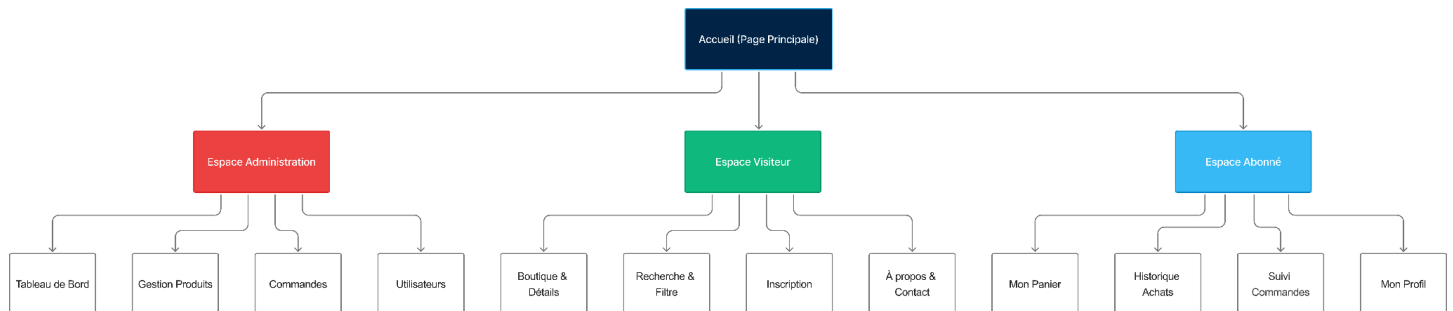
Chaque application répond à un besoin spécifique et technique :

- La Médiathèque propose une interface utilisateur moderne avec un thème sombre, souvent appelé Dark Mode. Son objectif est d'offrir une recherche rapide et fluide de médias grâce à des requêtes asynchrones.
- MaBoutique est une plateforme e-commerce complète. Elle garantit des transactions sécurisées à travers un tunnel d'achat clair et propose un tableau de bord Back-Office pour l'administration des ventes et des produits.
- L'API Cahier de Recettes permet de gérer des données culinaires de façon centralisée. Elle est conçue pour être consommée par différents clients, comme des applications web ou mobiles, avec un haut niveau de sécurité basé sur une authentification par token.
- L'API Cahier de Recettes permet de gérer des données culinaires de façon centralisée. Elle est conçue pour être consommée par différents clients (applications web ou mobiles) avec un haut niveau de sécurité (authentification par token).

2. CAHIER DES CHARGES ET SPÉCIFICATIONS

2.1. Expression des besoins

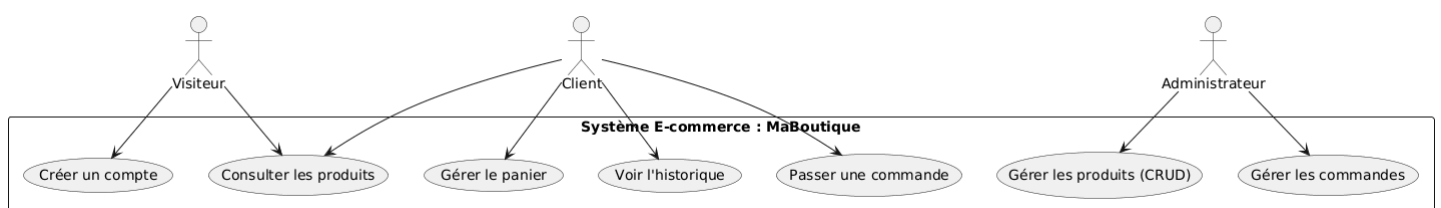
Pour le projet principal, MaBoutique, le besoin initial était de développer une plateforme e-commerce complète et fonctionnelle. Nous avons réalisé ce projet en équipe. L'application doit permettre aux clients de consulter et d'acheter des produits facilement sur tous les écrans grâce à une approche Responsive Design. De son côté, le propriétaire doit disposer d'un outil simple et sécurisé pour gérer son catalogue de produits et suivre les commandes.



2.2. Rôles des utilisateurs

Afin de structurer l'application et de sécuriser les accès, j'ai défini trois rôles principaux :

- Le Visiteur : Il peut naviguer sur le site, consulter le catalogue et utiliser la barre de recherche. Cependant, il doit obligatoirement créer un compte de manière sécurisée pour finaliser un achat.
- Le Client connecté : Il dispose d'un espace personnel sécurisé. Il peut gérer son panier d'achat, valider sa commande via le tunnel d'achat et consulter l'historique de ses achats récents.
- L'Administrateur : Il possède un accès exclusif au tableau de bord Back-Office. Il a les droits nécessaires pour gérer les produits avec les opérations classiques de création et de modification, modifier le statut des commandes et administrer les comptes utilisateurs.



2.3. Les User Stories (MVP)

Afin de définir le Produit Minimum Viable, ou MVP, et de planifier le développement selon la méthodologie Agile, j'ai rédigé des scénarios utilisateurs. Cette approche permet de comprendre précisément les besoins et les objectifs de chaque type d'utilisateur. Le diagramme ci-dessous illustre les fonctionnalités principales développées pour la plateforme :

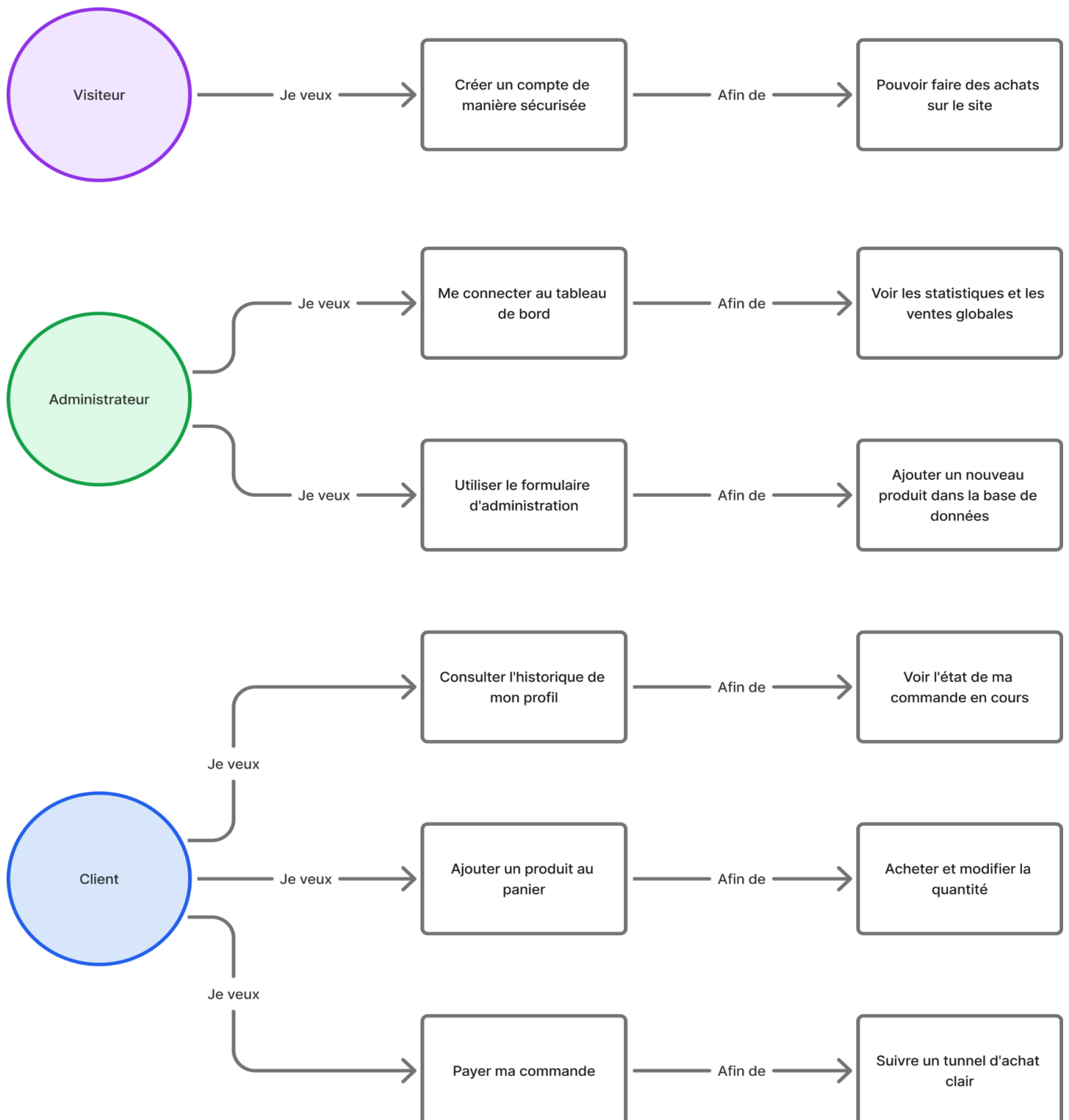


Diagramme des User Stories (MVP) illustrant les rôles et les objectifs des utilisateurs.

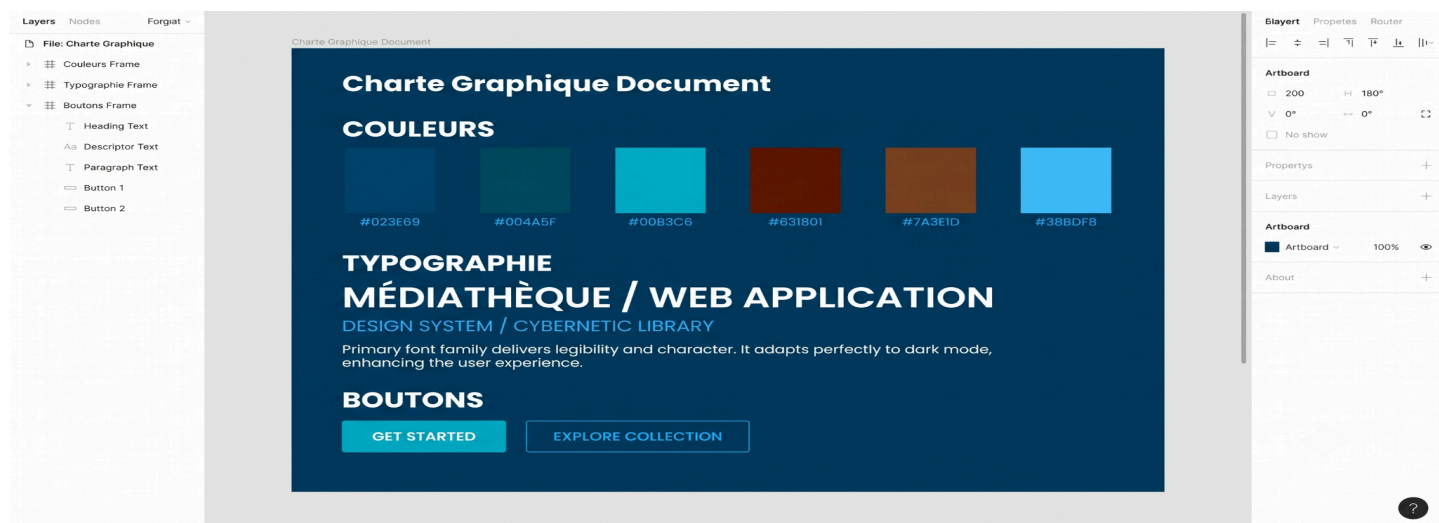
PARTIE 1 : MÉDIATHÈQUE DE PARIS (FOCUS FRONT-END - CCP 1)

3. CONCEPTION ET INTÉGRATION UI/UX

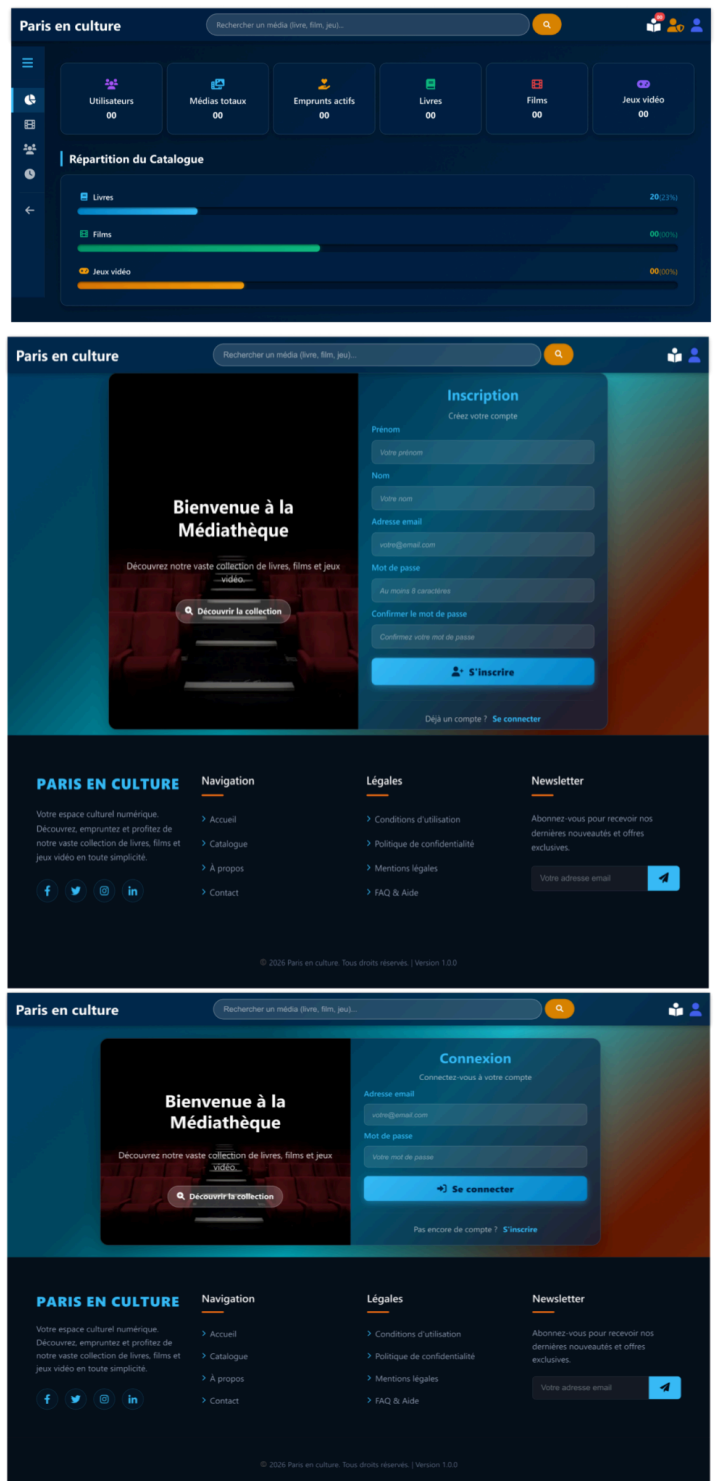
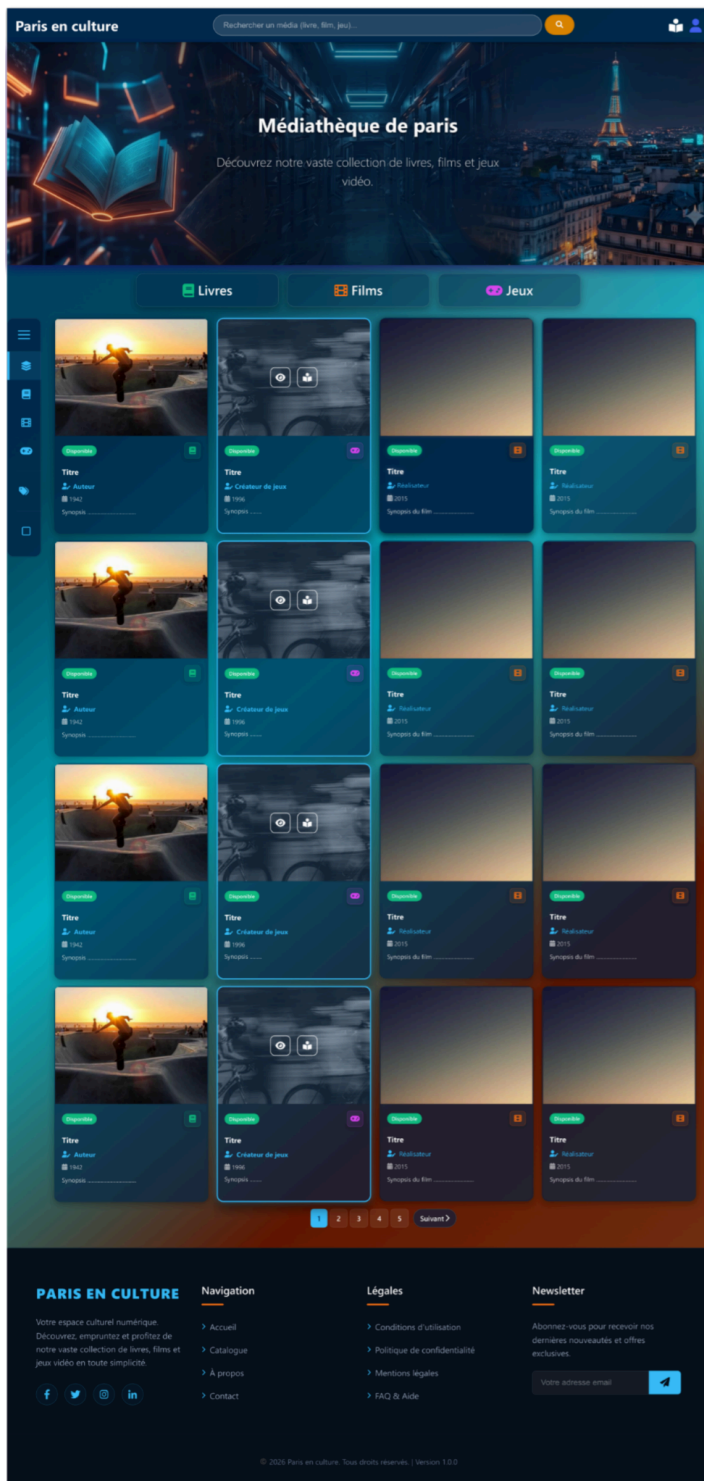
3.1. Maquettes et charte graphique

L'objectif de ce projet d'étude était de concevoir une interface utilisateur moderne et intuitive. J'ai choisi un design sombre, appelé Dark Mode, avec une esthétique futuriste et cyberpunk. Les couleurs principales sont un fond violet très sombre. J'ai ajouté des touches de cyan néon et de magenta. Cela crée un fort contraste visuel.

Avant de coder, j'ai utilisé l'outil Figma pour réaliser des maquettes haute fidélité. Cette étape de préparation m'a permis de valider la charte graphique. Elle m'a aussi aidé à bien structurer la navigation et à optimiser l'expérience utilisateur globale.



Charte graphique et composants UI réalisés sur Figma.

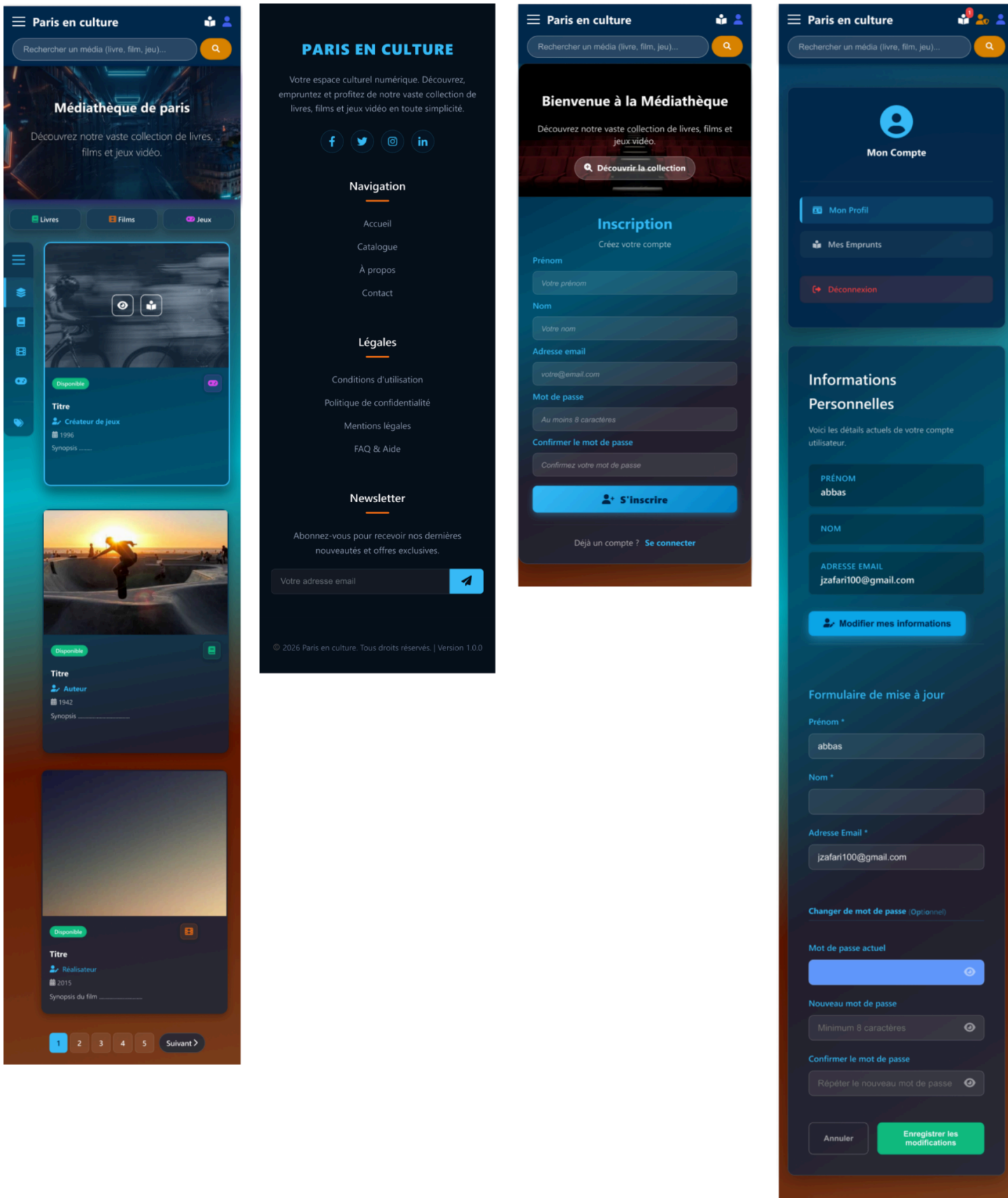


3.2. Intégration et Responsive Design

Aujourd'hui, la majeure partie de la navigation sur le web se fait avec un smartphone. C'est pour cette raison que j'ai adopté la méthode Mobile-First au début de l'intégration.

J'ai écrit le code CSS avec des Media Queries. Cela permet d'adapter le design dynamiquement à toutes les résolutions d'écran. Ainsi, l'interface reste toujours fluide, lisible et parfaitement organisée. L'expérience utilisateur est optimale, que la personne navigue

sur un grand écran d'ordinateur de bureau, sur une tablette ou sur un petit téléphone mobile.

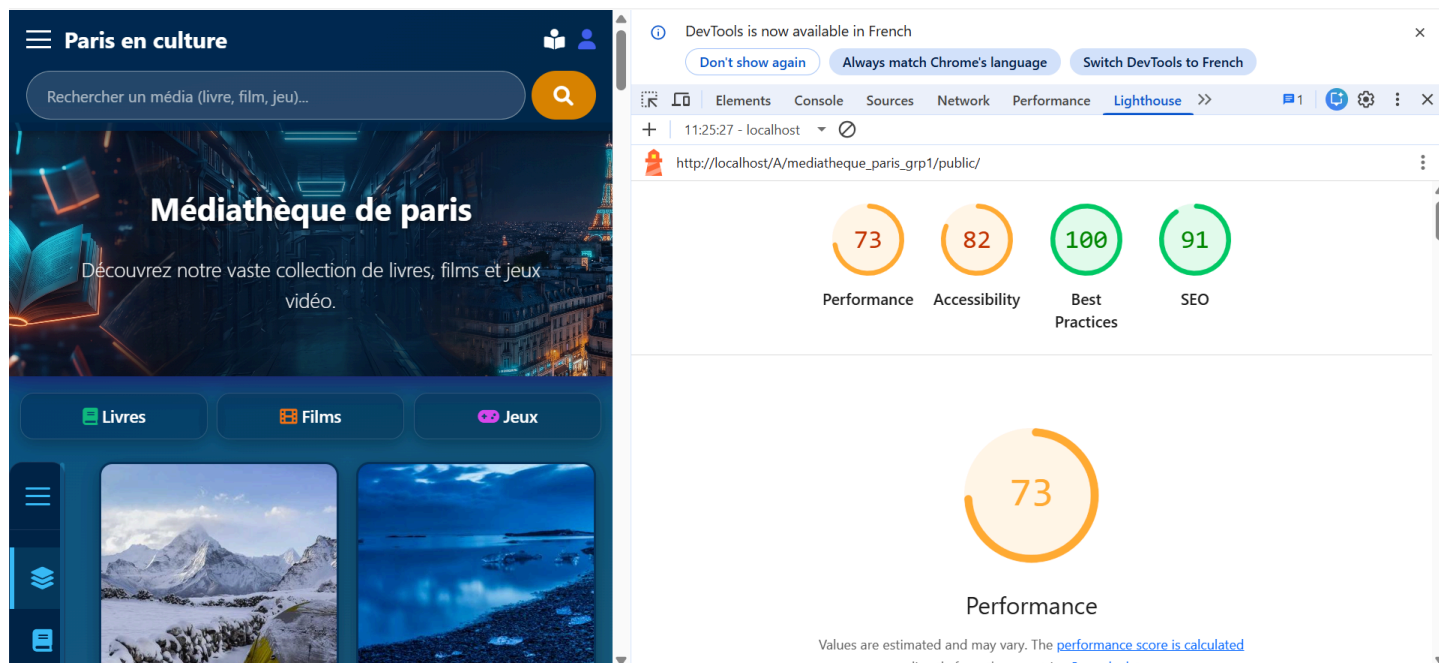


Démonstration du Responsive Design (approche Mobile-First)

3.3. Accessibilité Web et SEO

Il est très important de garantir que l'application soit utilisable par tous et bien indexée par les moteurs de recherche. Pour cela, j'ai appliqué les normes W3C et les directives d'accessibilité WCAG :

- Accessibilité visuelle et technique : Le choix du thème sombre offre un ratio de contraste optimal. Cela permet de réduire la fatigue des yeux. De plus, j'ai ajouté un attribut HTML descriptif, appelé attribut alt, sur toutes les images. Ce texte aide les logiciels de lecture d'écran à lire le contenu pour les personnes malvoyantes.
- Sémantique HTML5 et référencement : J'ai structuré le code avec des balises sémantiques claires. J'ai utilisé des balises comme header, main et section à la place des simples balises div. J'ai aussi respecté rigoureusement la hiérarchie des titres sur toutes les pages. Ces bonnes pratiques, combinées avec les balises meta, améliorent le référencement naturel ou SEO, et augmentent la qualité technique du site web.



Audit Lighthouse validant les performances, l'accessibilité et le SEO.

4. DYNAMISME ET JAVASCRIPT

4.1. Requêtes asynchrones et recherche en direct

Pour rendre mon application beaucoup plus interactive, j'ai développé plusieurs scripts en JavaScript natif. Mon objectif principal était d'ajouter une fonctionnalité de recherche en direct, souvent appelée Live Search.

Quand un utilisateur tape le nom d'un livre ou d'un film, le système utilise la technologie Fetch. Cette interface moderne permet d'envoyer des requêtes asynchrones vers la base de données. Le serveur cherche les informations et renvoie les résultats très rapidement.

Grâce à cette méthode, l'application affiche les résultats de manière instantanée juste sous la barre de recherche. L'avantage majeur est qu'il n'est pas du tout nécessaire de recharger toute la page web. Cela réduit fortement le temps d'attente pour le client. En fin de compte, cette fonctionnalité optimise les performances globales du site et rend la navigation beaucoup plus agréable.



Interface de la recherche en direct et extrait du code JavaScript utilisant l'API Fetch.

```
1 document.addEventListener('DOMContentLoaded', function() {
11   searchInput.addEventListener('input', function() {
12     clearTimeout(timeout);
13     const query = this.value.trim();
14
15     if (query.length < 2) {
16       resultsBox.style.display = 'none';
17       resultsBox.innerHTML = '';
18       return;
19     }
20
21     timeout = setTimeout(() => {
22       fetch(`${apiUrl}?q=${encodeURIComponent(query)}`)
23         .then(response => {
24           if (!response.ok) throw new Error("Erreur réseau");
25           return response.json();
26         })
27         .then(data => {
28           resultsBox.innerHTML = '';
29
30           if (data.length === 0) {
31             resultsBox.innerHTML = '<div style="padding: 15px; color: #94a3b8; text-align: center; font-size: 14px;">Aucun résultat trouvé</div>';
32           } else {
33             data.forEach(item => {
34               const itemUrl = `${baseUrl}/${item.id}`;
35               const imgSrc = item.image_url ? item.image_url : 'assets/images/default.jpg';
36               const html = `
37
```

Fonctionnement de la recherche en direct (Live Search) via l'API Fetch.

4.2. Interactions utilisateur et notifications

Il est très important d'informer l'utilisateur après chaque action sur le site. Pour répondre à ce besoin, j'ai créé un système de notifications dynamiques. On appelle souvent cela des alertes Toast. J'ai programmé cette fonctionnalité entièrement en JavaScript.

Par exemple, si un client confirme l'emprunt d'un livre, un petit message s'affiche en haut de l'écran. Cette alerte n'est pas gênante pour la lecture. Elle reste visible pendant quelques secondes, puis elle disparaît toute seule grâce à un minuteur automatique.

Ce mécanisme est très utile. Il donne un retour visuel direct pour indiquer si l'action est un succès ou s'il y a une erreur. Cela donne un aspect très professionnel au projet. Le confort de navigation est amélioré parce que l'utilisateur sait toujours ce qui se passe sur la plateforme.

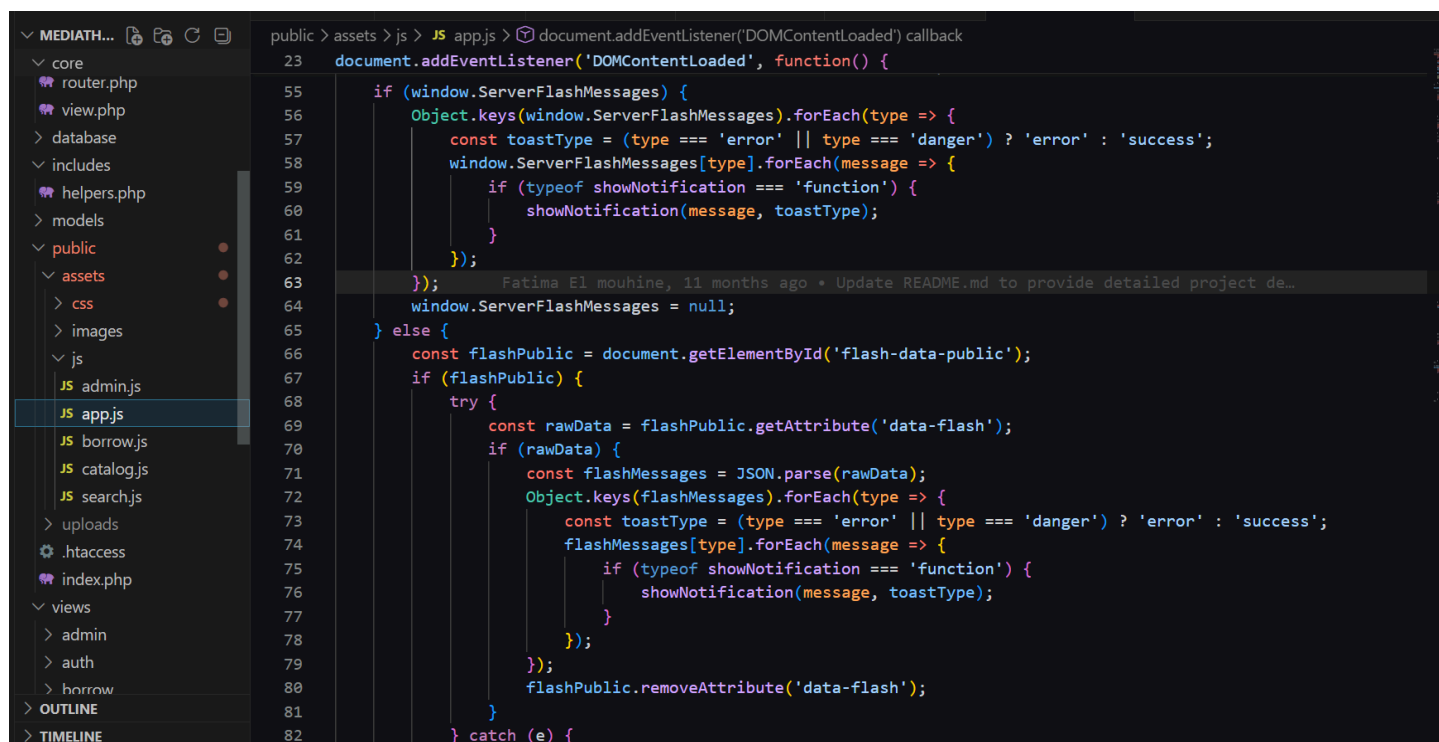
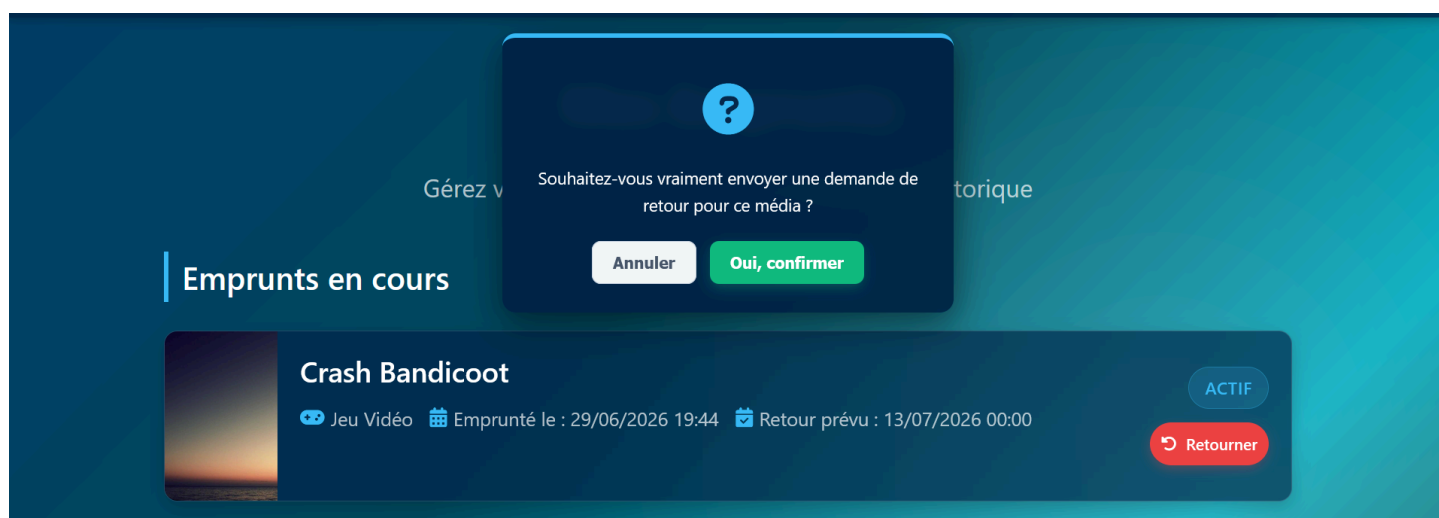


Figure X : Notification "Toast" et gestion des événements du DOM en JavaScript.

PARTIE 2 : MABOUTIQUE (FOCUS BACK-END ET SÉCURITÉ - CCP 2)

5. ARCHITECTURE ET BASE DE DONNÉES

5.1. L'architecture MVC en PHP natif

Pour démontrer mes compétences en développement Back-End, j'ai développé le site e-commerce MaBoutique en PHP natif. J'ai mis en place l'architecture Modèle Vue Contrôleur, souvent appelée architecture MVC. Cette structure professionnelle sépare rigoureusement la logique métier, la gestion des données et l'interface utilisateur :

- Le Modèle gère les données et exécute les requêtes SQL avec la base de données MySQL.
- La Vue gère l'affichage visuel pour l'utilisateur avec du code HTML, CSS et JavaScript.
- Le Contrôleur intercepte les requêtes de l'utilisateur, interroge le Modèle et charge la bonne Vue.

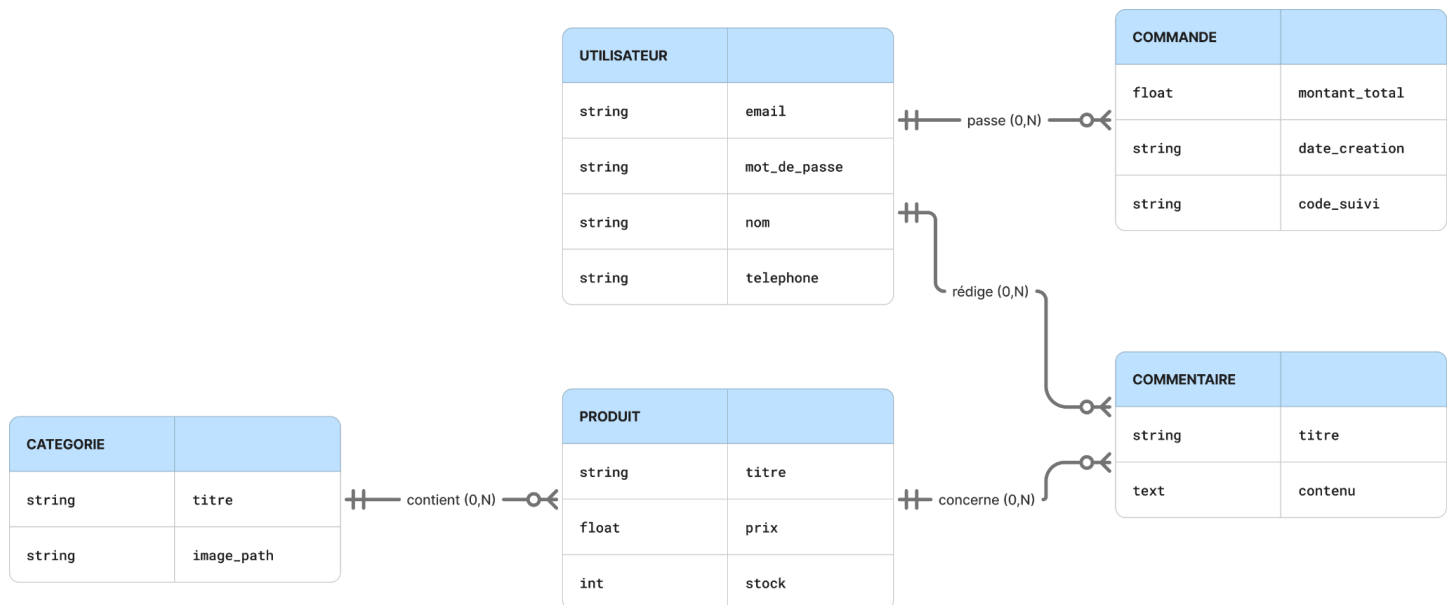
Toutes les requêtes passent par un routeur central. Cela garantit un code propre, bien sécurisé et très facile à maintenir.

5.2. Modélisation des données avec MySQL

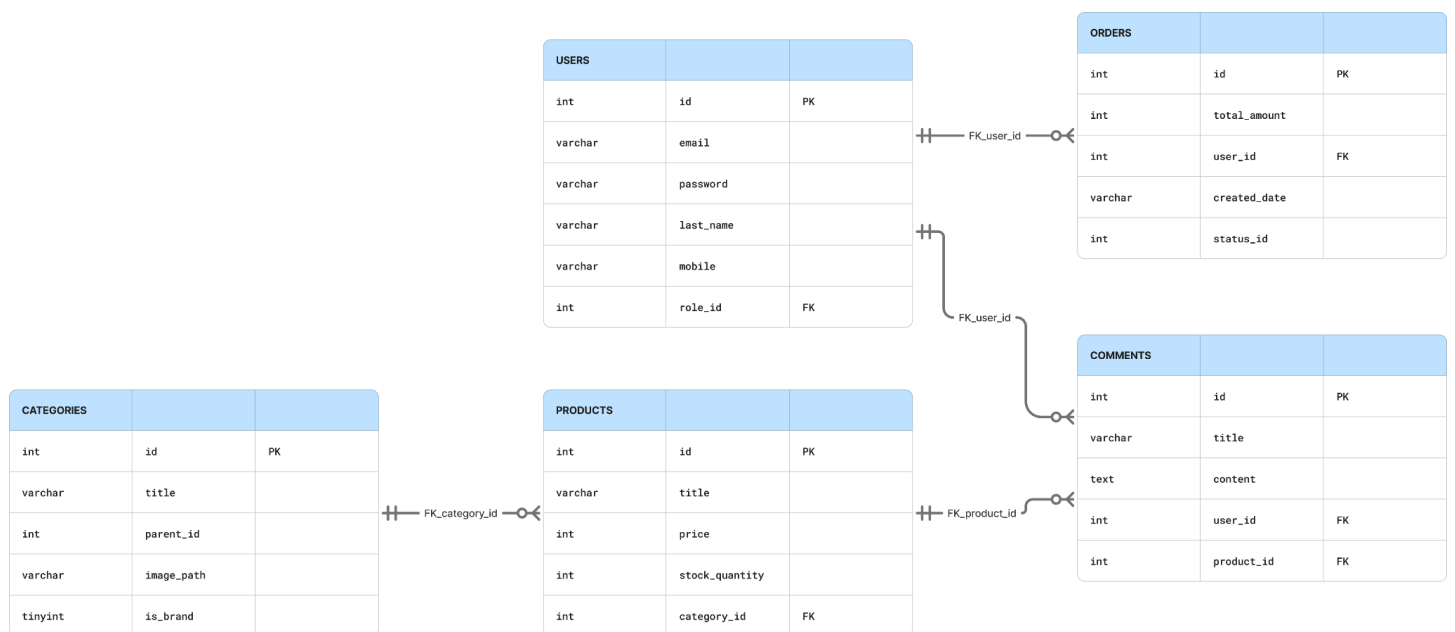
La gestion des données repose sur une base de données relationnelle MySQL. Pour garantir la cohérence des informations, j'ai d'abord conçu un Modèle Conceptuel de Données, ou MCD. Ensuite, j'ai créé un Modèle Logique de Données, ou MLD. Les tables principales sont liées par des clés étrangères pour bien structurer les informations.

Les tables principales sont :

- La table des utilisateurs pour la gestion sécurisée des comptes clients et des administrateurs.
- Les tables des produits et des catégories pour la gestion du catalogue d'articles.
- Les tables des commandes et des détails de commandes pour le suivi de l'historique et du statut des achats.



Modèle Conceptuel de Données (MCD) illustrant les entités principales de MaBoutique, leurs propriétés et leurs cardinalités (0,N / 1,1)



Modèle Conceptuel de Données (MCD) et Modèle Logique de Données (MLD) illustrant la structure de MaBoutique.

5.2.1. Dictionnaire de Données

Afin de documenter précisément la structure de la base de données, j'ai élaboré un dictionnaire de données. Ce document facilite la maintenance future du projet par d'autres développeurs. Il précise le type, la taille et le rôle exact de chaque champ pour les tables principales.

ORDERS			
int	id	PK	Identifiant Unique
int	user_id	FK	Client_Acheteur
int	total_amount		Montant_Total
int	status_id		État_De_Commande
varchar	tracking_code		Suivi_Postal
datetime	created_date		Date_Validation

USERS			
int	id	PK	Identifiant Unique
varchar	email		Pour_Connexion
varchar	password		Haché_Sécurisé
int	role_id	FK	1_Admin_3_Client
varchar	mobile		Numéro_Contact
timestamp	created_at		Date_Création

PRODUCTS			
int	id	PK	Identifiant Unique
varchar	title		Nom_Du_Produit
int	price		Prix_En_Centimes
int	stock_quantity		Stock_Actuel
int	category_id	FK	Catégorie_Liée
int	discount_percent		Réduction_Promo

Dictionnaire de données des tables principales (Users, Products, Orders) précisant le type et le rôle de chaque champ.

5.3. Connexion sécurisée avec PDO

La sécurité de la base de données est une priorité absolue dans mon projet. Pour interagir avec la base MySQL, j'utilise l'extension native PHP Data Objects, ou PDO.

Afin de neutraliser totalement la faille critique d'Injection SQL, toutes les requêtes sont exécutées via des requêtes préparées. Ainsi, les données saisies par les utilisateurs ne sont jamais insérées directement dans le code SQL. Le système prépare la requête en premier, puis il exécute les données séparément pour bloquer les codes malveillants.

```
core > model.php > Model > __construct
4 class Model
9 public function __construct()
11 // Inclusion de la configuration d'environnement sécurisée
12 $envPath = __DIR__ . '/env.php';
13 if (file_exists($envPath)) {
14     require_once $envPath;
15 } else {
16     die("Erreur de sécurité : Le fichier core/env.php est introuvable. Veuillez le créer à partir
17     de core/env.example.php.");
18 }
19 // Récupération des constantes d'environnement
20 $servername = defined('DB_HOST') ? DB_HOST : 'localhost';
21 $username = defined('DB_USER') ? DB_USER : 'root';
22 $password = defined('DB_PASS') ? DB_PASS : '';
23 $dbname = defined('DB_NAME') ? DB_NAME : 'digi_mvc';
24
25 $initCommand = defined('Pdo\Mysql::ATTR_INIT_COMMAND') ? \Pdo\Mysql::ATTR_INIT_COMMAND :
26 \PDO::MYSQL_ATTR_INIT_COMMAND;
27 // Options PDO sécurisées
28 $attr = array(
29     $initCommand => "SET NAMES utf8mb4",
30     PDO::ATTR_EMULATE_PREPARES => false,
31     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
32 );
33
34 if (self::$conn === null) {
35     try {
```

```
core > model.php > Model
4 class Model
85 // Exécute une requête SELECT avec des requêtes préparées
86
87 public function doSelect($sql, $values = array(), $fetch = '', $fetchStyle = PDO::FETCH_ASSOC)
88 {
89     $stmt = self::$conn->prepare($sql);
90     foreach ($values as $key => $value) {
91         $stmt->bindValue($key + 1, $value);
92     }
93     $stmt->execute();
94
95     if ($fetch == '') {
96         $result = $stmt->fetchAll($fetchStyle);
97     } else {
98         $result = $stmt->fetch($fetchStyle);
99     }
100     return $result;
101 }
102
103 // Exécute une requête INSERT, UPDATE ou DELETE sécurisée
104
105 public function doQuery($sql, $values = array())
106 {
107     $stmt = self::$conn->prepare($sql);
108     foreach ($values as $key => $value) {
109         $stmt->bindValue($key + 1, $value);
110     }
111     $stmt->execute();
112 }
```

Configuration sécurisée de la connexion PDO - Utilisation de requêtes préparées pour prévenir les injections SQL.

6. FONCTIONNALITÉS ET ADMINISTRATION

6.1. Le tunnel d'achat et la gestion du panier

Pour offrir une expérience d'achat très fluide et sécurisée sur la plateforme MaBoutique, nous avons développé un parcours utilisateur clair en plusieurs étapes. On appelle souvent ce processus le tunnel d'achat.

Dans un premier temps, nous avons programmé la gestion du panier. L'utilisateur peut ajouter de nouveaux produits, modifier les quantités souhaitées ou supprimer des articles à tout moment. Les calculs mathématiques, comme les sous-totaux et le prix total avec les taxes, sont mis à jour de manière dynamique.

Ensuite, il y a la validation de la commande. Une fois connecté à son compte, le client doit confirmer son adresse de livraison et choisir son mode de paiement. Après ces étapes, il valide sa commande. Le système affiche un récapitulatif clair avant la confirmation finale pour éviter les erreurs.

The screenshot displays the MaBoutique checkout interface. At the top, the MaBoutique logo is on the left, and a search bar with the placeholder 'Rechercher un produit...' is in the center. To the right of the search bar are icons for user profile, wishlist (2 items), and shopping cart (3 items). Below the header is a navigation menu with categories: TOUT, SMARTPHON, ORDINATEURS PORTABLE, ACCESSOIRE, AUDIO & ÉCOUTEURS, APPLE, SAMSUNG, SONY, and XIAOMI. The main content area is divided into two columns. The left column shows the checkout progress: 'Connexion', 'Livraison' (active), and 'Paiement'. Under 'Livraison', there are two steps: 1. 'Choisissez une adresse de livraison' with a '+ Ajouter une adresse' button, and 2. 'Choisissez le mode de livraison' with two options: 'Livraison Express (24h) 5,00 €' and 'Livraison Standard Gratuit'. The right column, titled 'Vos articles (3)', lists the items in the cart: HP Envy x360 (1400,00 € / un.), AirTag Apple (35,00 € / un.), and Support Ordinateur Alu1 (45,00 € / un.). Below the items, a summary shows 'Sous-total 1480,00 €' and 'Remise - 6,75 €'. The final 'Total TTC' is 1473,25 €. A blue button at the bottom right says 'Procéder au paiement →'.

Article	Quantité	Prix unitaire	Prix total
HP Envy x360	1	1400,00 € / un.	1400,00 €
AirTag Apple	1	35,00 € / un.	35,00 €
Support Ordinateur Alu1	1	45,00 € / un.	45,00 €
Sous-total			1480,00 €
Remise			- 6,75 €
Total TTC			1473,25 €

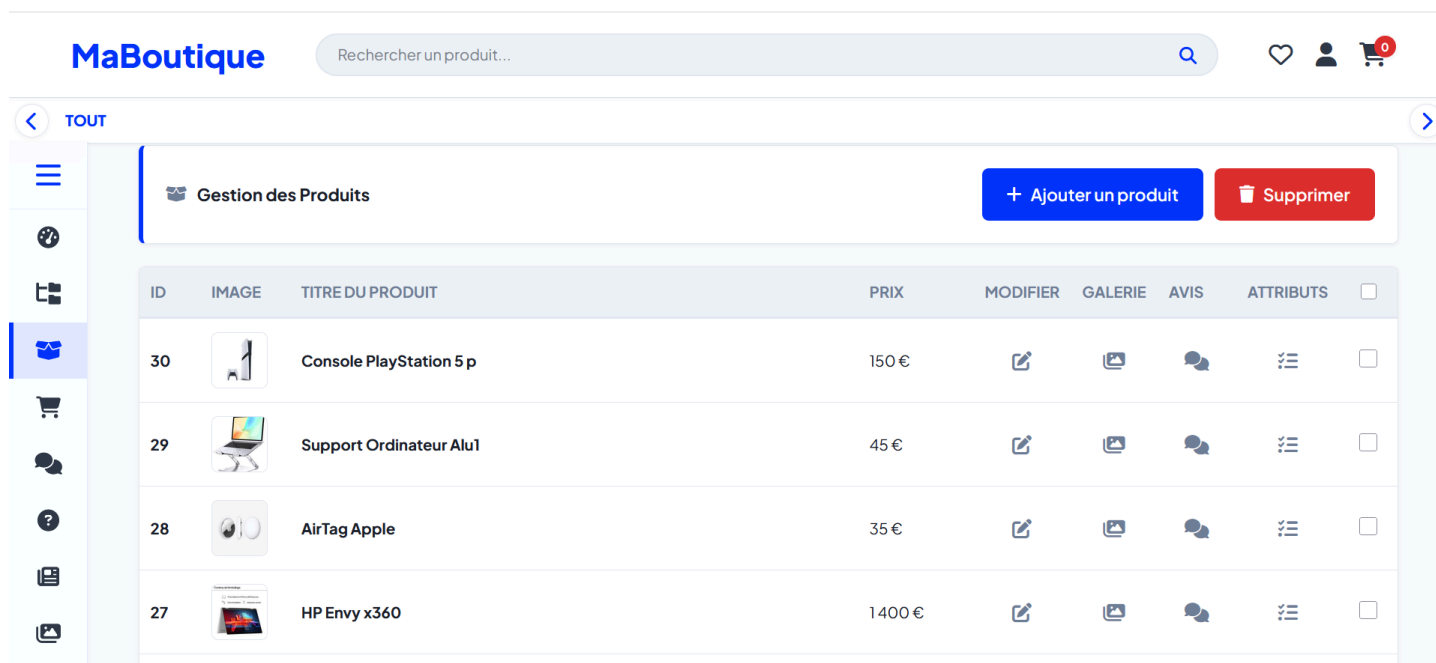
Déroulement du tunnel d'achat et confirmation de commande.

6.2. Panneau d'administration et opérations CRUD

Pour gérer le site e-commerce de manière totalement autonome, nous avons développé une interface d'administration sécurisée, aussi appelée tableau de bord ou Back-Office. L'accès à cet espace privé est strictement réservé aux utilisateurs qui possèdent le rôle d'administrateur dans la base de données.

Depuis ce tableau de bord, l'administrateur peut réaliser toutes les opérations de gestion des données. Ces actions sont regroupées sous l'acronyme CRUD, ce qui signifie Créer, Lire, Mettre à jour et Supprimer.

Par exemple, nous avons créé un contrôleur spécifique pour les produits. Grâce à ce fichier technique, l'administrateur peut ajouter un nouvel article, consulter la liste du catalogue, modifier un prix de vente ou supprimer un produit obsolète. De plus, cet espace permet de suivre efficacement toutes les commandes des clients et de modifier leur statut d'expédition, comme en attente, validée ou expédiée.



Interface du panneau d'administration (Back-Office) illustrant la gestion des produits (opérations CRUD).

7.2. Authentification et gestion des sessions

La protection des comptes utilisateurs est une priorité absolue. Les mots de passe ne sont jamais stockés en texte clair dans la base de données. Ils sont cryptés avec la fonction PHP `password_hash`. Cette fonction utilise un algorithme très robuste pour transformer le mot de passe en une chaîne de caractères illisible. Lors de la connexion, le système vérifie la correspondance grâce à la fonction native `password_verify`.

Pour maintenir la connexion de l'utilisateur de manière sécurisée, nous gérons les sessions PHP. L'identifiant de la session est régénéré régulièrement, par exemple après une connexion réussie. Cela permet d'empêcher le vol de session. Enfin, un système de contrôle vérifie le rôle de l'utilisateur sur chaque page. Cela bloque l'accès au tableau de bord pour toute personne non autorisée et la renvoie vers la page d'accueil.

```
models > ModelLogin.php > ModelLogin > checkUser
8 class ModelLogin extends Model
20 public function checkUser(array $form): bool
28 {
29 }
30 {
31 // Prévention de l'injection SQL grâce aux requêtes préparées PDO
32 $sql = "SELECT id, password FROM users WHERE email = ?";
33 $user = $this->doSelect($sql, [$email], 'fetch', PDO::FETCH_ASSOC);
34 {
35 // Vérification du mot de passe haché
36 if ($user && password_verify($password, $user['password'])) {
37     Model::sessionInit();
38
39 // Régénération de l'ID de session contre la fixation de session
40 session_regenerate_id(true);
41
42 // Forçage du typeage en entier
43 Model::sessionSet('userId', (int)$user['id']);
44 Model::sessionSet('loggedIn', true);
45
46 return true;
47 }
48
49 return false;
50 }
51 }
52 >>
```

```
models > ModelRegister.php > ModelRegister > insertUser
3 class ModelRegister extends Model
11 public function insertUser(array $data): bool
18 {
19 $lastName = htmlspecialchars(trim($data['last_name'] ?? ''), ENT_QUOTES, 'UTF-8');
20 $mobile = htmlspecialchars(trim($data['mobile'] ?? ''), ENT_QUOTES, 'UTF-8');
21 $newsletter = isset($data['newsletter']) ? 1 : 0;
22 {
23 // Hachage robuste du mot de passe avec l'algorithme par défaut (bcrypt)
24 $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
25
26 // Valeurs par défaut pour le profil d'un nouvel inscrit
27 $roleId = 3; // 3 correspond au rôle client/utilisateur standard
28 $nationalId = '';
29 $phone = '';
30 $birthDate = '';
31 $address = '';
32 $gender = 1;
33 $username = '';
34 $city = '';
35 $postalCode = '';
36 {
37 // Gestion de la date
38 $createdAt = date('Y-m-d H:i:s');
39 if (method_exists($this, 'getCurrentDate')) {
40     $createdAt = self::getCurrentDate('Y-m-d H:i:s');
41 }
42 {
43 // Vérification de l'existence de l'e-mail pour éviter les doublons
44 $sqlCheck = "SELECT id FROM users WHERE email = ?";
45 $result = $this->doSelect($sqlCheck, [$email], 'fetch', PDO::FETCH_ASSOC);
```

Sécurisation de l'authentification avec le hachage des mots de passe et la gestion des sessions.

7.3. Conformité RGPD et Protection des Données

Nous avons intégré plusieurs principes fondamentaux pour protéger la vie privée des utilisateurs. Il est obligatoire de respecter le Règlement Général sur la Protection des Données, ou RGPD :

- Minimisation des données : Lors de l'inscription, nous demandons seulement les informations strictement nécessaires pour la facturation et la livraison, comme le nom et l'adresse. Aucune donnée superflue ou sensible n'est stockée dans la base.
- Droit à l'oubli : Dans son espace personnel, le client a la possibilité de consulter toutes ses informations. Il peut les mettre à jour, ou demander la suppression définitive de son compte et de toutes ses données personnelles.
- Gestion des cookies : Les cookies utilisés sur le site sont uniquement des cookies techniques. Ils sont strictement nécessaires au fonctionnement du panier d'achat et de la session.

The screenshot displays the 'MaBoutique' user interface. At the top, there's a navigation bar with the brand name 'MaBoutique' and a search bar. Below this is a category menu with options like 'TOUT', 'SMARTPHONE', 'ORDINATEURS PORTABLE', etc. The main content area is titled 'Client(e) de la boutique' and features a sidebar with links to 'Tableau de bord', 'Mes commandes', 'Mes favoris', 'Mes réductions', and 'Mes informations' (which is highlighted). The 'Mes informations' section contains form fields for 'Mobile' (with the number 0602234983), 'Adresse', 'Ville', and 'Code postal'. There are two prominent blue buttons: 'Mettre à jour le profil' at the bottom of the form and 'Changer le mot de passe' in a separate box on the right. The bottom of the sidebar includes links for 'Déconnexion' and 'Supprimer le compte'.

Illustration de la conformité RGPD sur l'interface utilisateur (gestion des données personnelles et droit à l'oubli).

7.4. Plan de Tests et Jeux d'Essai

Afin de garantir la robustesse de l'application avant sa mise en production, nous avons élaboré un plan de tests manuels rigoureux. Ces jeux d'essai couvrent les fonctionnalités critiques, la logique métier et la sécurité du site.

Nous avons simulé le parcours d'un vrai client. Par exemple, nous avons essayé de créer un compte avec un email qui existe déjà dans la base de données. Nous avons aussi essayé d'accéder aux pages privées de l'administrateur avec un compte client normal pour vérifier les redirections de sécurité.

Enfin, nous avons testé la logique du panier d'achat, notamment quand un client demande une quantité supérieure au stock disponible. Le tableau ci-dessous montre les résultats détaillés de nos tests principaux :

TEST_3_LOGIQUE_PANIER		
Text	Scenario	Panier avec Quantite superieure au stock
Text	Action_Attendue	Ajustement au max disponible et Alerte
Text	Resultat_Final	✔ Succes

TEST_1_SECURITE		
Text	Scenario	Inscription avec un email deja existant
Text	Action_Attendue	Bloquer insertion et afficher erreur
Text	Resultat_Final	✔ Succes

TEST_2_DROITS_ACCES		
Text	Scenario	Acces a l URL Back-Office par un Client
Text	Action_Attendue	Redirection vers accueil (Erreur 403)
Text	Resultat_Final	✔ Succes

Extrait du plan de tests (Jeux d'essai) validant la sécurité et la logique métier de l'application MaBoutique.

8. GESTION DE PROJET ET VERSIONING

8.1. Workflow avec Git et GitHub

Pour travailler en équipe, nous avons utilisé les outils Git et GitHub. Cela permet de sécuriser le code et de bien collaborer. Nous avons utilisé la méthode des branches (Feature Branching).

Pour chaque nouvelle tâche, nous avons créé une branche séparée. À la fin de la tâche, nous avons fait une "Pull Request" pour envoyer le code sur la branche principale ("main"). Cette méthode évite les conflits entre les développeurs et garde un historique très propre.

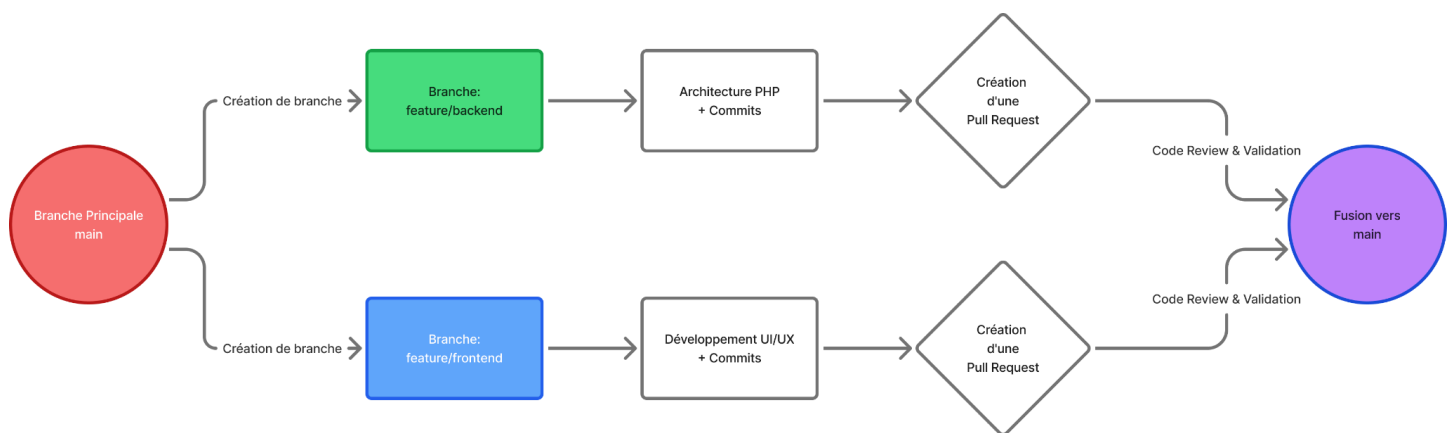


Schéma du workflow Git (Feature Branching) utilisé pour la collaboration en équipe.

8.2. Organisation du travail en équipe

La réussite de nos projets repose sur une bonne communication. Nous avons divisé les tâches selon les compétences de chacun.

- Pour le projet Médiathèque : J'ai travaillé spécifiquement sur le développement Front-End (Design, intégration et JavaScript) en collaboration avec un autre membre du groupe.
- Pour le projet MaBoutique : J'ai travaillé sur le développement Back-End (Architecture MVC, PHP et base de données MySQL) avec plusieurs autres membres de l'équipe.

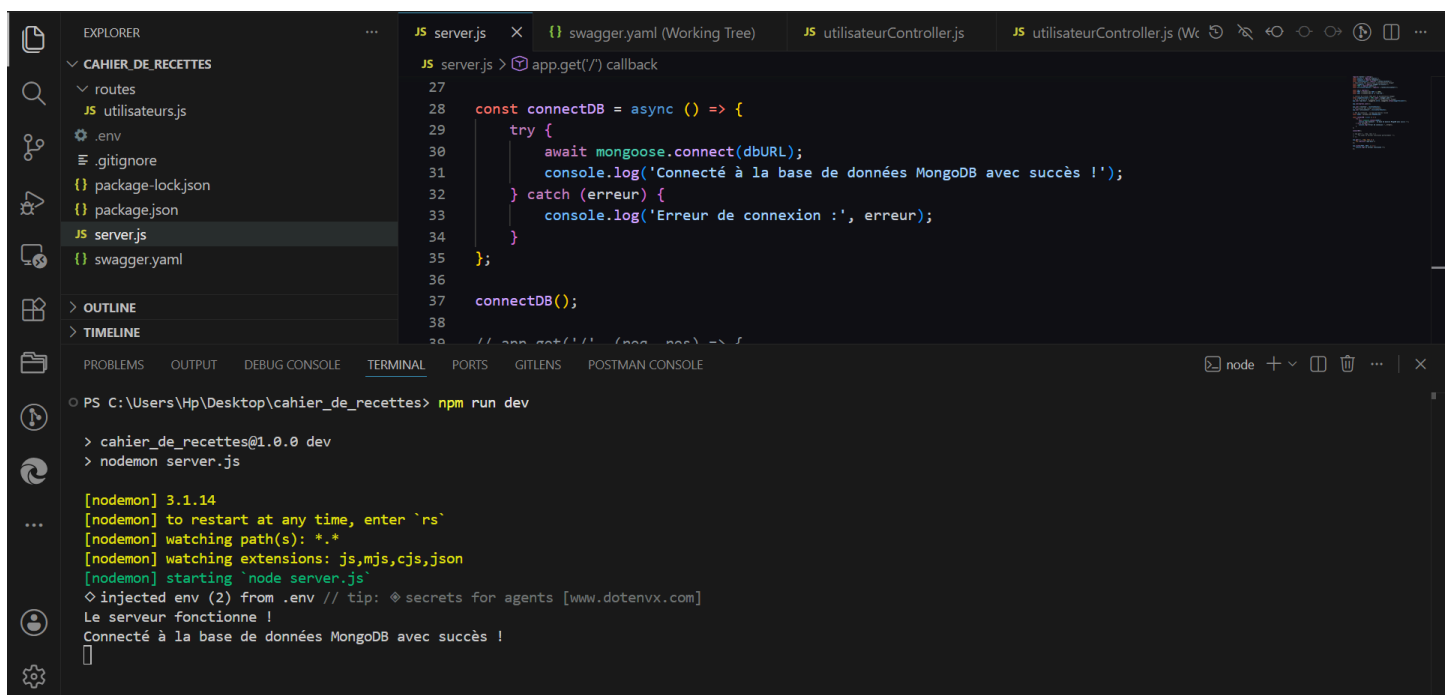
Nous avons organisé des réunions fréquentes. Cela nous a permis de valider notre architecture technique, de synchroniser notre code et de corriger les bugs ensemble. Ce travail de groupe a beaucoup amélioré mes compétences.

9. CAHIER DE RECETTES COLLABORATIF

9.1. Présentation du projet

Pour mon troisième projet, j'ai conçu et développé une application intitulée Cahier de Recettes. Il s'agit d'une API RESTful. J'ai créé ce projet avec l'environnement d'exécution Node.js et le framework Express.js.

L'objectif principal de ce projet d'étude était de maîtriser le développement d'un Back-End moderne en JavaScript. Je voulais aussi assurer une gestion très sécurisée des utilisateurs et manipuler une base de données non relationnelle, souvent appelée base de données NoSQL.



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays the project structure for 'CAHIER_DE_RECETTES', including files like .env, package-lock.json, package.json, server.js, and swagger.yaml. The main editor area shows the 'server.js' file with a code snippet for connecting to MongoDB using Mongoose. The terminal at the bottom shows the command 'npm run dev' being executed, and the output indicates that the server is running successfully and connected to the MongoDB database.

```
JS server.js X {} swagger.yaml (Working Tree) JS utilisateurController.js JS utilisateurController.js (Wk ...
EXPLORED
CAHIER_DE_RECETTES
  routes
  JS utilisateurs.js
  .env
  .gitignore
  {} package-lock.json
  {} package.json
  JS server.js
  {} swagger.yaml
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS POSTMAN CONSOLE
PS C:\Users\Hp\Desktop\cahier_de_recettes> npm run dev
> cahier_de_recettes@1.0.0 dev
> nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ◇ secrets for agents [www.dotenvx.com]
Le serveur fonctionne !
Connecté à la base de données MongoDB avec succès !
```

Démarrage du serveur Node.js et connexion réussie à la base de données MongoDB.

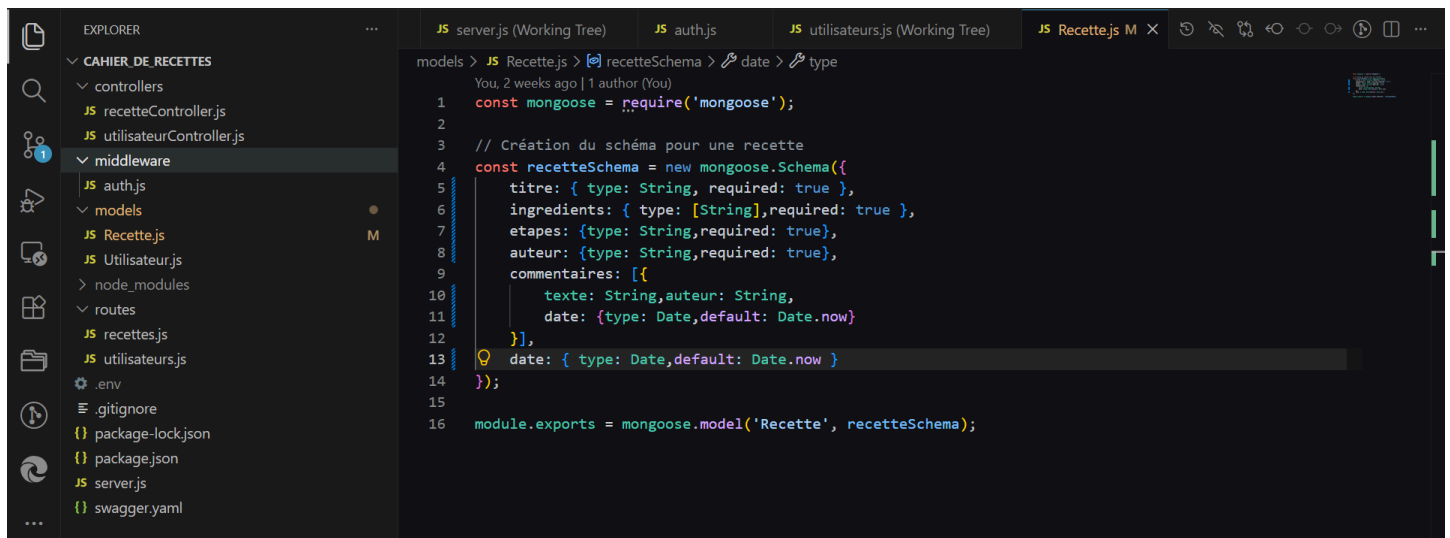
9.2. Architecture et Base de données avec MongoDB

Pour la gestion des données, j'ai choisi MongoDB. C'est une base de données orientée documents. Afin de bien structurer les données et d'interagir facilement avec la base, j'ai utilisé la bibliothèque technique Mongoose. L'architecture de l'application repose sur deux modèles principaux :

- Le modèle Utilisateur : Il gère les comptes des membres avec une adresse email unique, un nom et un mot de passe sécurisé.

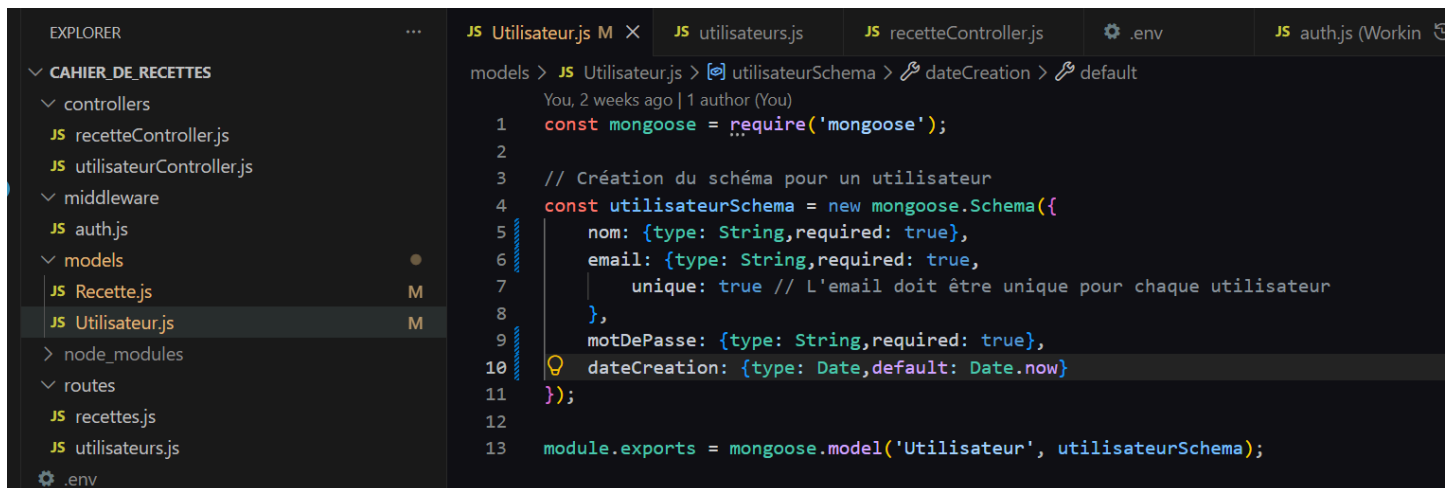
- Le modèle Recette : Il permet de stocker le titre du plat, la liste des ingrédients, les étapes de préparation et le nom de l'auteur. Il contient aussi un tableau pour stocker les commentaires des utilisateurs.

Ce choix technique apporte une grande flexibilité. Il offre aussi d'excellentes performances pour le traitement des données au format JSON.



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer is expanded to the 'models' folder, showing 'Recette.js' and 'Utilisateur.js'. The main editor window displays the code for 'Recette.js'. The code defines a Mongoose schema for a recipe, including fields for title, ingredients, steps, author, and a comments array. The schema is then registered with Mongoose.

```
models > JS Recette.js > [M] recetteSchema > [M] date > [M] type
You, 2 weeks ago | 1 author (You)
1 const mongoose = require('mongoose');
2
3 // Création du schéma pour une recette
4 const recetteSchema = new mongoose.Schema({
5   titre: { type: String, required: true },
6   ingredients: { type: [String], required: true },
7   etapes: { type: String, required: true },
8   auteur: { type: String, required: true },
9   commentaires: [{
10     texte: String, auteur: String,
11     date: { type: Date, default: Date.now }
12   }],
13   date: { type: Date, default: Date.now }
14 });
15
16 module.exports = mongoose.model('Recette', recetteSchema);
```



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer is expanded to the 'models' folder, showing 'Recette.js' and 'Utilisateur.js'. The main editor window displays the code for 'Utilisateur.js'. The code defines a Mongoose schema for a user, including fields for name, email, password, and creation date. The schema is then registered with Mongoose.

```
models > JS Utilisateur.js > [M] utilisateurSchema > [M] dateCreation > [M] default
You, 2 weeks ago | 1 author (You)
1 const mongoose = require('mongoose');
2
3 // Création du schéma pour un utilisateur
4 const utilisateurSchema = new mongoose.Schema({
5   nom: { type: String, required: true },
6   email: { type: String, required: true,
7     unique: true // L'email doit être unique pour chaque utilisateur
8   },
9   motDePasse: { type: String, required: true },
10  dateCreation: { type: Date, default: Date.now }
11 });
12
13 module.exports = mongoose.model('Utilisateur', utilisateurSchema);
```

Définition des schémas Mongoose pour les collections Utilisateur et Recette.

9.3. Sécurité et Authentification avec les jetons JWT

La sécurité de l'interface de programmation, ou API, est garantie par plusieurs mécanismes très rigoureux. Lors de l'inscription d'un nouvel utilisateur, son mot de passe est haché avec la bibliothèque technique bcrypt. Ensuite, il est enregistré de manière sécurisée dans la base de données MongoDB.

Pour l'authentification lors de la connexion, j'ai mis en place un système de jetons sécurisés. On appelle cela JSON Web Tokens, ou JWT. Quand un utilisateur se connecte avec succès, le serveur lui donne un jeton signé.

Ensuite, j'ai développé un fichier de contrôle, appelé middleware d'autorisation. Ce programme intercepte toutes les requêtes sur les routes protégées, comme l'ajout, la modification ou la suppression d'une recette. Il vérifie si le jeton JWT est présent et valide. Si le jeton est faux ou absent, le système bloque automatiquement l'accès avec une erreur d'autorisation.

```
CAHIER DE RECETTES
├── controllers
│   ├── recetteController.js
│   └── utilisateurController.js
├── middleware
│   └── auth.js
├── models
│   ├── Recette.js
│   └── Utilisateur.js
├── node_modules
├── routes
│   ├── recettes.js
│   └── utilisateurs.js
├── .env
├── .gitignore
├── package-lock.json
├── package.json
├── server.js
└── swagger.yaml

middleware > JS auth.js > ...
You, 2 weeks ago | 1 author (You)
const jwt = require('jsonwebtoken');
const CLE_SECRETE = process.env.CLE_SECRETE;

module.exports = (req, res, next) => {
  try {
    // Récupérer le token depuis l'en-tête de la requête
    const enTeteAuth = req.headers.authorization;

    if (!enTeteAuth) {
      return res.status(401).json({ message: 'Accès refusé. Veuillez vous connecter.' });
    }

    // Séparer le mot "Bearer" du vrai token
    const token = enTeteAuth.split(' ')[1];

    // Vérifier si le token est valide
    const tokenDecode = jwt.verify(token, CLE_SECRETE);

    // Ajouter les informations de l'utilisateur à la requête
    req.utilisateur = tokenDecode;

    next();
  } catch (erreur) {
    res.status(401).json({ message: 'Token invalide ou expiré.' });
  }
};
```

Code du middleware d'authentification

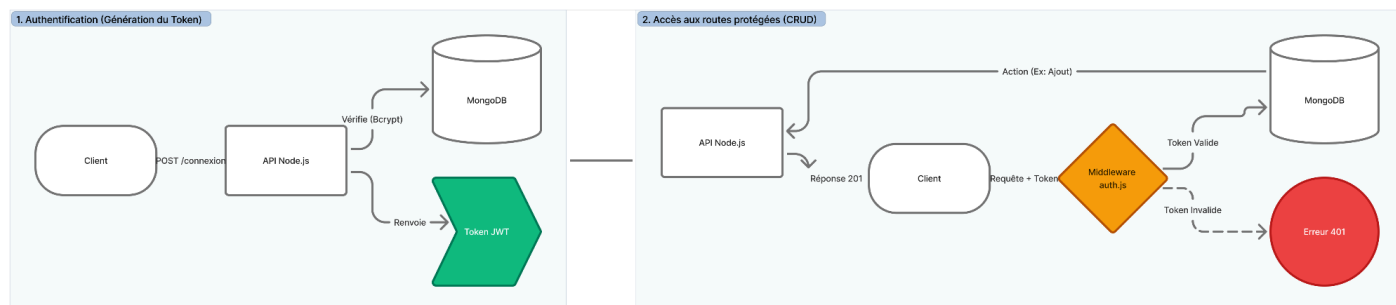
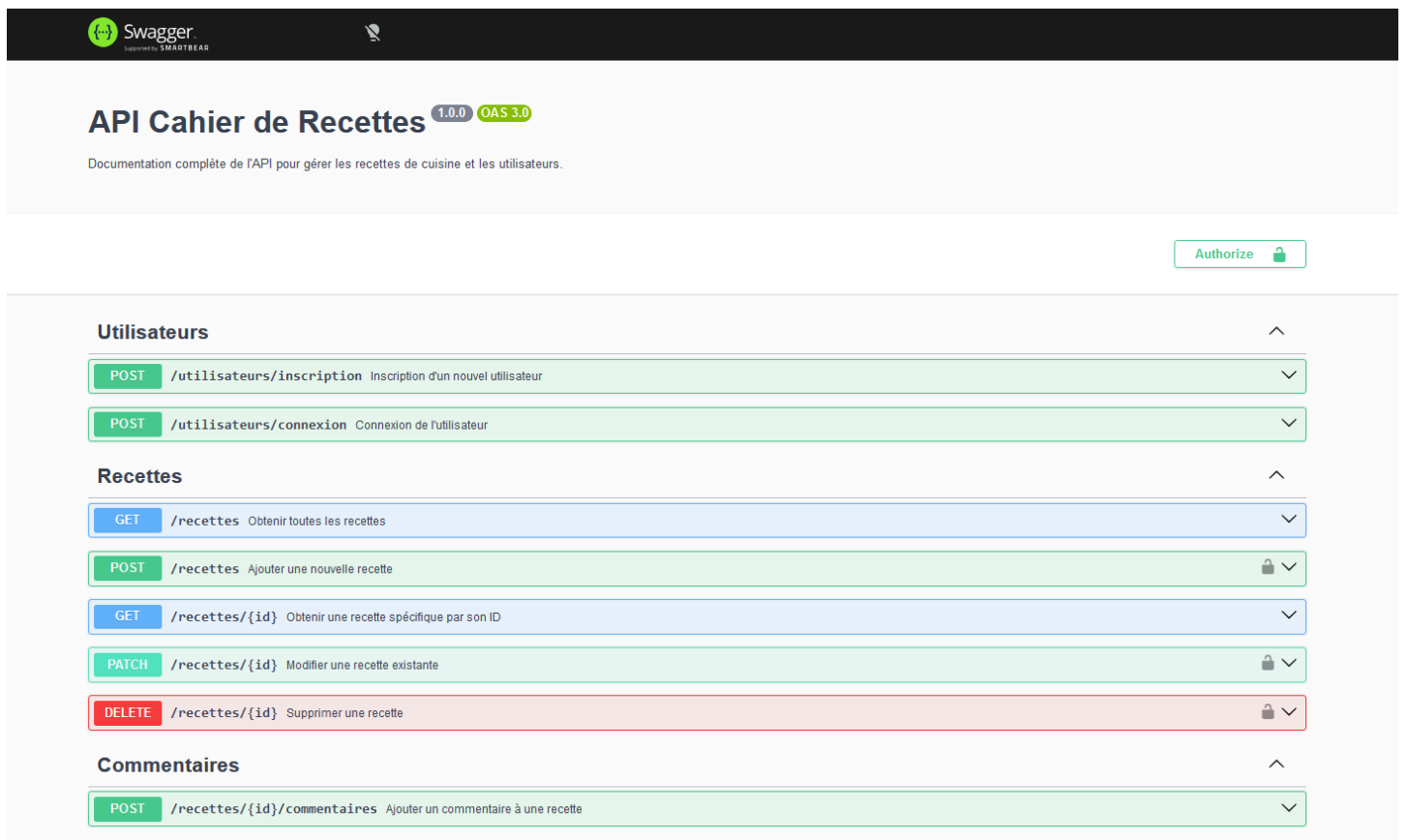


Diagramme de séquence illustrant le processus d'authentification et d'autorisation via JWT.

9.4. Documentation technique avec Swagger

Il est très important de rendre l'API professionnelle et facile à utiliser par d'autres développeurs ou par une application Front-End. Pour cela, j'ai rédigé une documentation complète avec l'outil Swagger, aussi connu sous le nom d'OpenAPI.

J'ai écrit cette documentation dans un fichier de configuration spécial. Elle explique en détail toutes les routes disponibles pour l'inscription, la connexion, et la gestion des recettes. L'interface graphique de Swagger permet de voir la structure exacte des données attendues. Elle offre aussi la possibilité de tester directement les requêtes HTTP depuis un simple navigateur web.



The screenshot displays the Swagger UI for the 'API Cahier de Recettes'. The header shows the Swagger logo and version information (1.0.0, OAS 3.0). The main content area lists the API endpoints, organized into three sections: 'Utilisateurs', 'Recettes', and 'Commentaires'. Each section contains a list of endpoints with their respective HTTP methods, paths, and descriptions. The 'Recettes' section is expanded, showing endpoints for GET, POST, PATCH, and DELETE operations. The 'Utilisateurs' section shows endpoints for POST operations. The 'Commentaires' section shows a POST endpoint. An 'Authorize' button is visible in the top right corner.

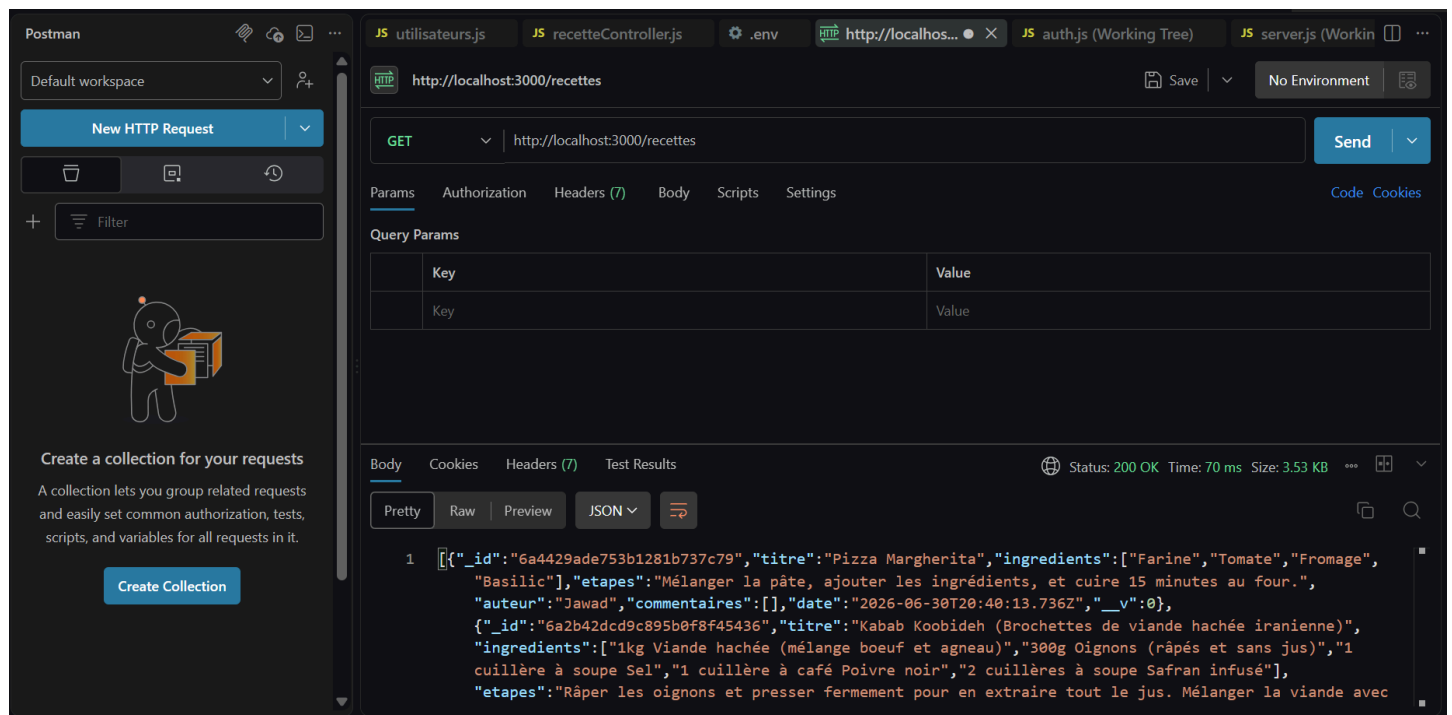
Resource	Method	Path	Description
Utilisateurs	POST	/utilisateurs/inscription	Inscription d'un nouvel utilisateur
	POST	/utilisateurs/connexion	Connexion de l'utilisateur
Recettes	GET	/recettes	Obtenir toutes les recettes
	POST	/recettes	Ajouter une nouvelle recette
	GET	/recettes/{id}	Obtenir une recette spécifique par son ID
	PATCH	/recettes/{id}	Modifier une recette existante
	DELETE	/recettes/{id}	Supprimer une recette
Commentaires	POST	/recettes/{id}/commentaires	Ajouter un commentaire à une recette

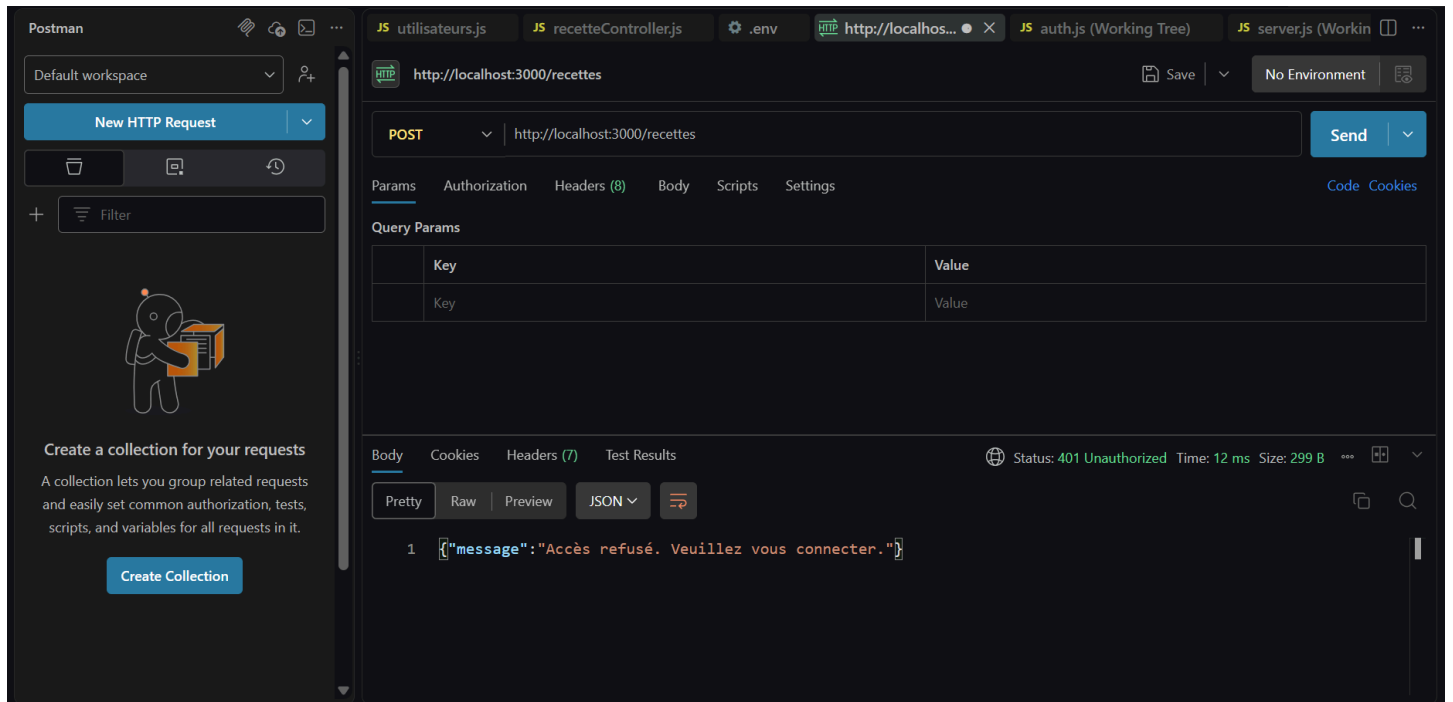
Documentation interactive OpenAPI / Swagger UI de l'API Cahier de Recettes.

9.5 - Phase de Tests et Validation avec Postman

Avant de valider l'API de manière définitive, j'ai exécuté une phase de tests systématiques. J'ai utilisé le logiciel Postman pour garantir la fiabilité globale du Back-End :

- Simulation des requêtes HTTP : J'ai créé une collection Postman qui regroupe toutes les routes. J'ai exécuté chaque méthode pour vérifier le bon fonctionnement des opérations de gestion sur le serveur Node.js.
- Vérification des codes de statut HTTP : J'ai contrôlé que le système renvoyait les bons codes de réponse. Par exemple, le serveur renvoie le code 201 pour une création réussie, le code 200 pour une simple lecture, et le code 401 pour un accès refusé.
- Gestion des erreurs : J'ai simulé des envois de données manquantes ou fausses, comme un email déjà existant. Le but était de m'assurer que l'API retournait des messages d'erreur très clairs au format JSON, sans bloquer ou arrêter le serveur.





Tests et validation des endpoints REST (codes de statut 200 et 401) via Postman.

10. CONCLUSION ET PERSPECTIVES

10.1. Bilan de la formation à La Plateforme

La réalisation de ces trois projets a été une étape très importante dans ma formation de Développeur Web et Web Mobile à La Plateforme. Ce parcours complet m'a permis de mettre en pratique toutes les compétences demandées pour le titre professionnel.

J'ai validé le CCP 1 pour la partie Front-End et le CCP 2 pour la partie Back-End. J'ai appris à concevoir des architectures techniques solides comme le modèle MVC et les API REST. J'ai aussi appris à sécuriser les données des utilisateurs et à créer des interfaces fluides qui s'adaptent à tous les écrans.

10.2. Compétences acquises

Durant ces mois de formation intensive, j'ai pu valider et pratiquer de nombreuses compétences clés:

- Apprentissage et mise en pratique de l'architecture MVC avec le langage PHP. J'ai compris comment bien séparer la logique et l'affichage.
- Capacité à créer des interfaces graphiques modernes et adaptables sur mobile à partir de maquettes réalisées sur l'outil Figma.
- Développement de composants dynamiques du côté client avec le langage JavaScript et la technologie Fetch pour les requêtes sans rechargement.
- Découverte et manipulation sécurisée de bases de données relationnelles avec MySQL et de bases de données non relationnelles avec MongoDB.
- Utilisation des outils de gestion de version comme Git et GitHub dans un cadre professionnel et pour le travail en équipe.

10.3. Évolutions futures des projets

Les bases techniques de nos applications sont solides, mais nous pouvons imaginer plusieurs améliorations. Pour le projet Médiathèque, nous pourrions intégrer un système de notifications automatiques par email. Cela permettrait de rappeler aux abonnés de rendre les livres avant la date limite.

Pour le projet Cahier de Recettes, l'étape suivante serait de développer une interface utilisateur complète en utilisant un framework moderne comme React. Cela permettrait de consommer l'API construite avec Node.js et d'offrir une meilleure expérience sur tous les écrans.

10.4. Déploiement et Mise en Production

Il était très important pour moi de ne pas rester seulement sur un environnement local. J'ai voulu concrétiser ces projets en les déployant sur des serveurs distants. Cela prouve ma capacité à comprendre la mise en production.

Les projets Médiathèque et MaBoutique sont hébergés sur un serveur personnel que j'administre avec le panneau de contrôle Plesk. L'API du Cahier de Recettes a été déployée avec succès sur la plateforme en ligne Alwaysdata. Tous ces projets, ainsi que la documentation technique, sont accessibles publiquement.

Voici les liens d'accès direct à mes réalisations :

- Médiathèque de Paris :  https://jawad-zafari.students-laplateforme.io/mediatheque_paris

- MaBoutique :  <https://jawad-zafari.students-laplateforme.io/maboutique>

- Cahier de Recettes, documentation interactive de l'API avec Swagger :

 <https://cahier-de-recettes.alwaysdata.net/api-docs>

- Portfolio Professionnel, centralisation de tous mes projets :

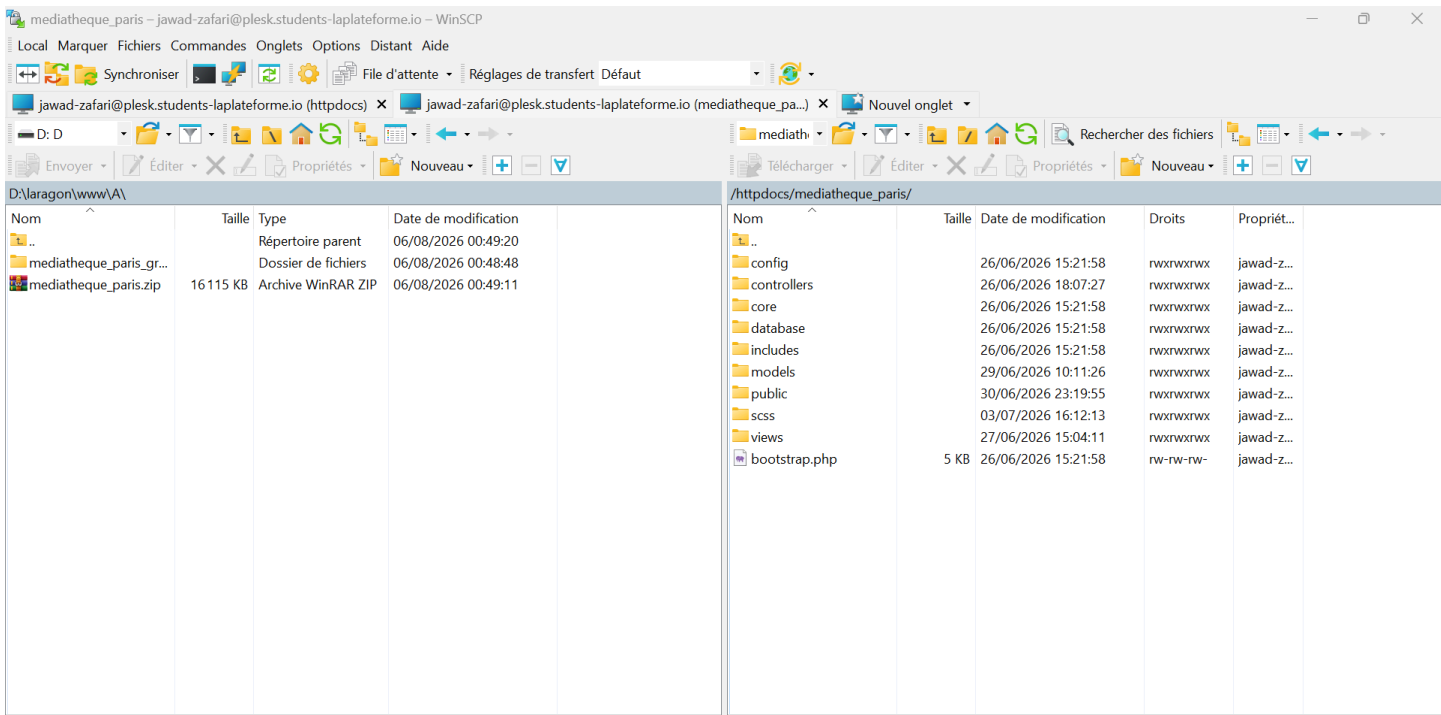
 <https://jawad-zafari.students-laplateforme.io>

10.5. Méthodologie de déploiement appliquée

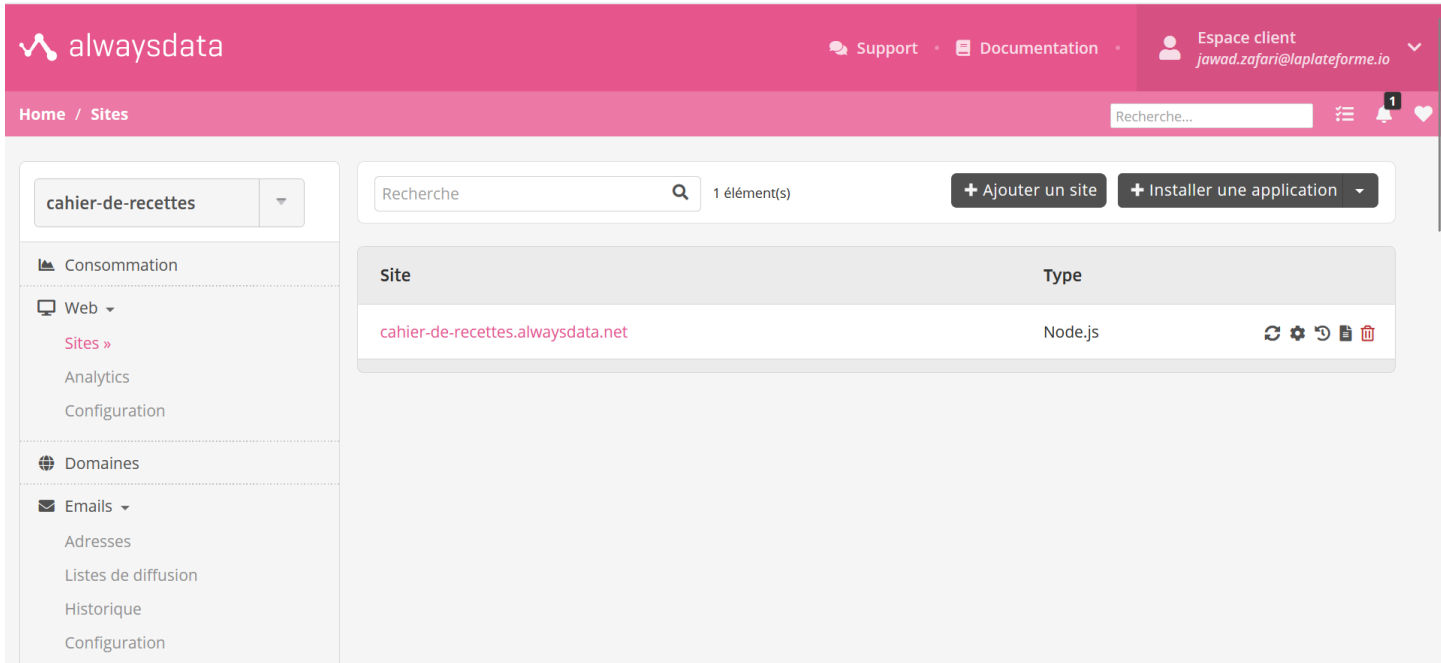
Pour mettre ces sites en ligne, j'ai appliqué une méthode stricte. Tout d'abord, j'ai utilisé le logiciel client FTP appelé WinSCP. Cet outil permet de transférer de manière sécurisée le code source depuis mon ordinateur vers les serveurs distants.

Sur le serveur Plesk, j'ai créé les bases de données de production. Ensuite, j'ai mis à jour les mots de passe de manière sécurisée dans les fichiers de configuration PHP. Sur la plateforme Alwaysdata, j'ai

accédé au terminal de commande. J'ai tapé les commandes nécessaires pour installer les dépendances du projet Node.js et j'ai configuré les variables d'environnement secrètes pour protéger l'API.



Transfert sécurisé des fichiers vers les serveurs distants via le client FTP WinSCP.

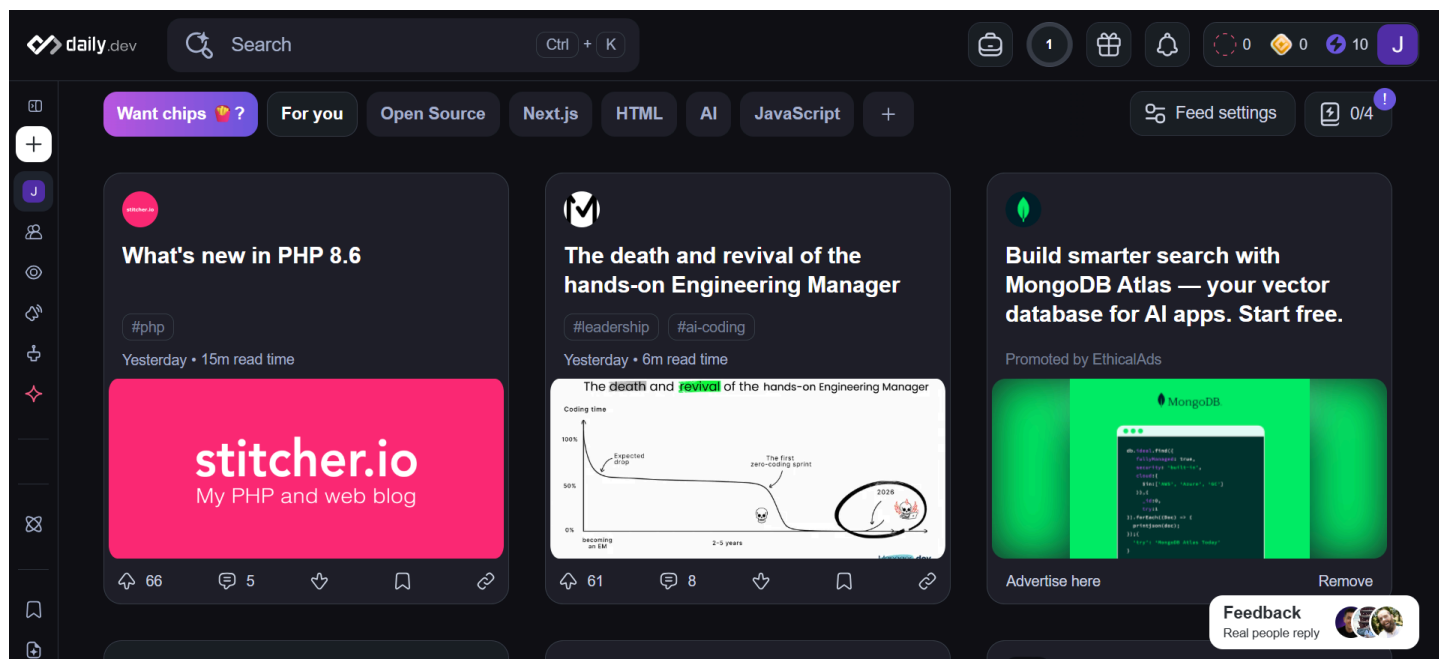


Interface Alwaysdata confirmant le déploiement et la configuration de l'API Node.js.

10.6. La Veille Technologique

Dans le monde du développement web, les technologies et les failles de sécurité changent très vite. Il est donc indispensable de garder ses connaissances à jour. J'ai mis en place une stratégie de veille technologique active pour continuer à apprendre.

J'utilise des outils comme Daily.dev pour regrouper les articles des meilleurs blogs techniques. Je consulte régulièrement des sites de référence comme MDN Web Docs ou Stack Overflow, surtout quand je cherche une bonne pratique. Je regarde aussi beaucoup de tutoriels en vidéo pour découvrir les nouveautés du langage PHP ou les règles de sécurité. Enfin, j'essaie de tester les nouvelles bibliothèques sur des petits projets personnels pour transformer la théorie en pratique.



Utilisation de l'extension Daily.dev pour la veille technologique quotidienne.